

SLOTIFY: A SMART APPOINTMENT BOOKING SYSTEM FOR SERVICE-BASED BUSINESSES

A Project Report

Submitted By

Aksh Thakkar - 2403031057002

Om Maurya - 2403031057048

Daksh Patel - 2403031057065

Jainil Patel - 2403031057071

in Partial Fulfilment For the Award of

the Degree of

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING

Under the Guidance of

Prof. Pratik Gite

Your Guide Designation



VADODARA

November - 2026



PARUL UNIVERSITY

CERTIFICATE

This is to Certify that Project - II (303105423) of 7th Semester entitled “Slotify: A Smart Appointment Booking System for Service-Based Businesses” of Group No. CSE_182 has been successfully completed by

- Aksh Thakkar - 2403031057002
- Om Maurya - 2403031057048
- Daksh Patel - 2403031057065
- Jainil Patel - 2403031057071

under my guidance in partial fulfillment of the Bachelor of Technology (B.Tech) in Computer Science and Engineering of Parul University in Academic Year 2026- 2027.

Date of Submission : _____

Prof. Pratik Gite

Prof. Namrata Patel

Dr. Shailendra K Mishra

Project Guide

Project Coordinator

Head of Department

CSE, PIET

CSE, PIET

CSE, PIET

Parul University

Parul University

Parul University

Acknowledgements

“The single greatest cause of happiness is gratitude.”

-Auliq-Ice

Acknowledgements

We express our sincere gratitude to all those who supported and guided us throughout the successful completion of this project. First and foremost, we are thankful to the Almighty for granting us the strength, wisdom, and determination to carry out this work.

We extend our heartfelt thanks to our project guide, **Prof. Pratik Gite**, for his invaluable guidance, continuous encouragement, and constructive feedback at every stage of this project. His expertise and dedication have been a constant source of inspiration. We are also thankful to the faculty members and staff of the Department of Computer Science and Engineering, Parul Institute of Engineering and Technology, Parul University, for providing us with the necessary resources, infrastructure, and academic support.

We sincerely thank **Dr. Shailendra K. Mishra**, Head of Department, CSE, PIET, Parul University, for his support and leadership. Finally, we are deeply grateful to our parents, family members, and well-wishers for their constant moral support and motivation throughout this journey.

AKSH THAKKAR

OM MAURYA

DAKSH PATEL

JAINIL PATEL

CSE, PIET

Parul University,

Vadodara

Abstract

Slotify is a smart appointment booking platform developed to simplify scheduling and improve service efficiency for modern businesses. The system provides a digital environment where customers can discover service providers, check real-time availability, book appointments, and receive automated confirmations. It is designed specifically for service-based businesses such as salons, clinics, and wellness centers where scheduling accuracy, transparency, and customer convenience are critical.

The project focuses on solving common issues such as double booking, missed appointments, manual scheduling conflicts, and poor customer engagement. Slotify integrates a dynamic slot generation mechanism, role-based access control, and streamlined booking workflows to improve user experience for customers, staff, and administrators. It also supports features such as booking confirmation, rescheduling, review management, and business-specific service handling.

In simple terms, the system helps customers book services quickly without calling or waiting in long queues, while also helping businesses manage their staff schedules more efficiently. Customers can choose the right service, see available time slots, and reserve an appointment in a few steps. This reduces confusion and saves time for both the user and the service provider.

The platform also gives business owners a better way to organize daily appointments, avoid scheduling mistakes, and keep a clear record of bookings. It improves communication between customers and providers and creates a more professional and reliable service experience. The project uses a web application with a user-friendly interface and a structured backend to manage data securely and efficiently.

By combining a web-based frontend with a robust backend and database-driven logic, Slotify offers an efficient and scalable solution for appointment management. The project demonstrates how modern technologies can be used to create a reliable and user-friendly scheduling system that reduces operational overhead while increasing trust and satisfaction for both businesses and customers.

Table of Contents

Acknowledgements	iii
Abstract	iv
List of Tables	ix
List of Figures	x
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Project Objectives	2
1.4 Scope	2
1.5 System Overview	2
1.6 Technology Stack	3
2 Literature Survey	5
2.1 Overview	5
2.2 Paper Reviews	6
2.3 Comparison of Existing Platforms	6
2.4 Reminder Effectiveness	7
2.5 Research Gap / What the Review Did Not Resolve	7

3	Software Requirements Specification (SRS)	9
3.1	Purpose and Scope	9
3.2	User Roles	9
3.2.1	Customer	9
3.2.2	Business Admin	9
3.2.3	Staff Member	9
3.2.4	Reception / Walk-In User	10
3.3	Functional Requirements	10
3.4	Non-Functional Requirements	10
3.5	Use Case Diagram	10
3.6	Constraints	10
4	System Design	13
4.1	Architecture Overview	13
4.2	Database Design	13
4.3	API Design	14
4.4	Booking Workflow	15
4.5	Security Design	15
4.6	UI Design Principles	16
5	Methodology	17
5.1	Development Approach	17
5.2	Phase 1: Requirements Gathering	17
5.3	Phase 2: System Design	17
5.4	Phase 3: Implementation	18
5.5	Phase 4: Testing	18
5.6	Tools and Environment	18

6	Implementation	20
6.1	Backend Implementation	20
6.1.1	Project Structure	20
6.1.2	Slot Availability Logic	20
6.1.3	Authentication Middleware	21
6.1.4	Email Notification Service	21
6.2	Frontend Implementation	21
6.2.1	Project Structure	21
6.2.2	Real-Time Slot Display	22
6.2.3	State Management and Prop Drilling Refactor	22
6.3	Implementation Status	23
7	Conclusion	25
7.1	Summary of Work Done	25
7.2	Objectives Met	25
7.3	Challenges Encountered	25
7.4	Key Takeaways	26
8	Future Work	27
8.1	What Comes Next	27
8.2	Payment Integration	27
8.3	Mobile Application	27
8.4	AI-Based Slot Recommendations	27
8.5	Advanced Analytics	28
8.6	Multi-Language Support	28
8.7	Multi-Branch Business Support	28

9	Research Paper / Survey Paper and Plagiarism Report	29
9.1	Research Paper	29
9.2	Plagiarism Report	38
	References	50

List of Tables

1.1	Key scheduling challenges and their solutions in Slotify	3
2.1	Feature Comparison: Slotify vs. Existing Platforms	7
3.1	Non-functional requirements for Slotify	11
4.1	Key design modules and responsibilities	15
4.2	Booking process stages in Slotify	15
5.1	Tools used during development	18
5.2	Development phases and expected outputs	19
6.1	Feature implementation status	24
7.1	Objective completion status	25

List of Figures

1.1	System overview of the Slotify appointment workflow	3
2.1	Overview of existing appointment scheduling challenges in service-based businesses	5
3.1	Use case diagram for Slotify user interactions and role-based workflow	12
4.1	System architecture of the Slotify booking platform	13
4.2	Database model and relationship flow for business, staff, and appointments	14
5.1	Methodology overview for the Slotify development lifecycle	17
6.1	Business configuration and setup screen	20
6.2	Business profile and service discovery interface	21
6.3	Landing page of the Slotify platform	22
6.4	Login interface for the user authentication flow	22
6.5	Admin dashboard showing bookings, activities, and business overview	23
6.6	Appointment list and scheduling records	23
6.7	Appointment lifecycle flow from booking to service completion	24

Chapter 1

Introduction

1.1 Background and Motivation

Most people who have tried to book an appointment at a local clinic or salon know the routine. A customer calls, waits on hold, confirms a time, writes it down, and hopes the other side remembers too. Sometimes the appointment goes smoothly, but often it is missed, double-booked, or forgotten because the process depends entirely on manual coordination.

This problem is common in small service businesses where operations are still largely managed through phone calls, paper registers, and informal memory. For example, a salon owner may keep a physical register of appointments, yet still face missed bookings, cancelled slots, or confusion over staff availability. Even when digital tools exist, many of them are too expensive, too complicated, or not designed for the needs of small local businesses.

The motivation for **Slotify** came from this gap in the market. Small service businesses need a simple, affordable, and reliable online system that helps customers book appointments quickly while letting service providers manage schedules more efficiently. The aim is to reduce manual work, avoid schedule conflicts, improve communication, and provide a smoother booking experience for both customers and businesses.

1.2 Problem Statement

Manual coordination overhead and appointment attrition are two sides of the same issue. When bookings are managed over phone calls or handwritten records, every transaction requires a human in the loop. Someone must confirm the time, record the booking, check for conflicts, and follow up with reminders. This works at low volume, but it breaks down as the business grows.

At the same time, no-shows and empty slots create revenue loss. Customers forget appointments, a cancellation is not re-filled quickly, or the business has no real-time visibility into scheduling gaps. Without reminders or automated slot management, small businesses lose time, money, and customer

trust.

Another major challenge is that many booking systems are designed for large organizations and do not fit the needs of small businesses such as salons, clinics, and wellness centres. These businesses need flexible calendars, staff assignments, and an easy process for managing appointments without unnecessary complexity.

1.3 Project Objectives

The main objectives of the project are as follows:

1. To build a customer-facing booking system that allows users to search for service providers, view available time slots, and book appointments without manual phone coordination.
2. To develop a provider dashboard for managing schedules, staff availability, and service timing efficiently.
3. To implement a real-time appointment mechanism that prevents double bookings and reduces schedule conflicts.
4. To provide automated confirmation and reminder emails so customers do not forget their appointments.
5. To create a simple and user-friendly interface for both customers and service providers.
6. To support business analytics such as appointment trends, cancellation rates, and workload distribution.

1.4 Scope

The current scope of the project covers service-based businesses that operate on a time-slot model, such as clinics, salons, tutoring services, and physiotherapy centres. The system is designed for multiple independent businesses, each with separate data and access controls.

The platform focuses on online appointment scheduling, business management, and automated notifications. Native mobile applications and in-app payment processing are outside the scope of the current phase, but they are considered as future enhancements.

1.5 System Overview

Slotify is a web-based platform that connects customers, service providers, and business administrators through a single scheduling system. Customers can browse services, check provider availability, and book appointments after selecting suitable time slots. Service providers can manage their working hours, assign staff, and monitor bookings from a dedicated dashboard.

The system is designed to handle the full booking lifecycle, from service discovery and slot selection to confirmation and reminder notifications. It also supports business-level data tracking, making it easier for owners to understand busy periods and improve operational decisions.

The key idea behind the system is to automate the scheduling process while keeping the interface simple and accessible. This reduces administrative effort and creates a more professional experience for customers.

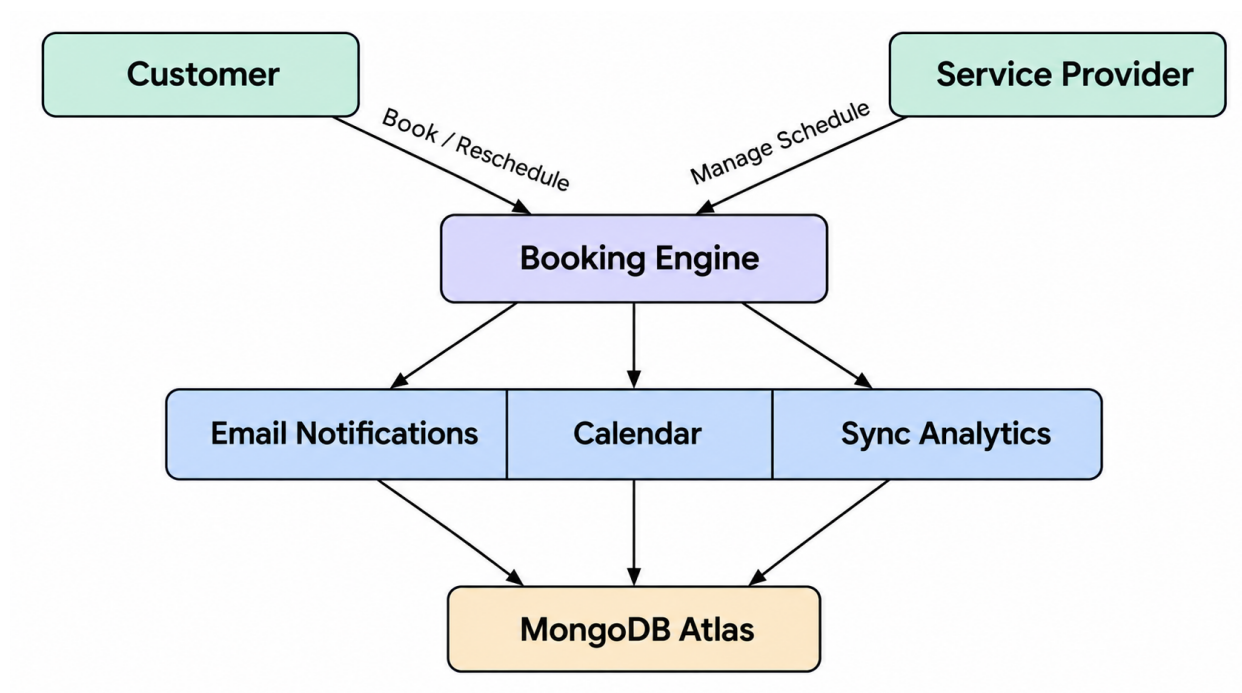


Figure 1.1: System overview of the Slotify appointment workflow

Table 1.1: Key scheduling challenges and their solutions in Slotify

Problem	How Slotify solves it
Double booking	Real-time slot checking and conflict prevention during appointment creation
Missed appointments	Confirmation emails and reminder notifications sent to customers
Manual coordination	Digital booking system reduces dependence on phone calls and paper records
Low business visibility	Dashboard tracks bookings, workload, and appointment trends

1.6 Technology Stack

The project is developed using the MERN stack, which provides a flexible and scalable foundation for building a web application. The stack includes:

- **MongoDB:** A NoSQL database used to store business details, users, appointments, services, and booking records.
- **Express.js:** A backend framework used to define APIs and manage server-side logic.
- **React.js:** A frontend library used to build the user interface and provide a responsive user experience.
- **Node.js:** A JavaScript runtime environment used for server-side execution.

The application also uses authentication mechanisms such as JWT (JSON Web Tokens) and Google OAuth for secure login and user verification. Appointment notifications are sent using Nodemailer, which enables real-time email confirmations and automated reminder messages. This combination of technologies allows the system to be scalable, secure, and easy to maintain.

In summary, **Slotify** addresses the scheduling challenges faced by small service businesses by delivering a digital, automated, and user-friendly appointment booking solution.

Chapter 2

Literature Survey

2.1 Overview

Appointment scheduling plays a central role in service-based businesses such as salons, clinics, beauty centers, repair shops, and wellness studios. In such businesses, each booking affects staff workload, customer satisfaction, and revenue generation. A poor scheduling system can create double bookings, missed appointments, idle time, and unnecessary manual work. As a result, scheduling is not only a convenience feature but also an operational need.

Many small businesses still rely on phone calls, manual calendars, or informal record keeping. Although this approach may work at a very small scale, it becomes difficult to manage as business size and customer demand increase. Literature in scheduling and operations management highlights that effective booking systems should reduce confusion, improve utilization, and support timely communication between customers and service providers.

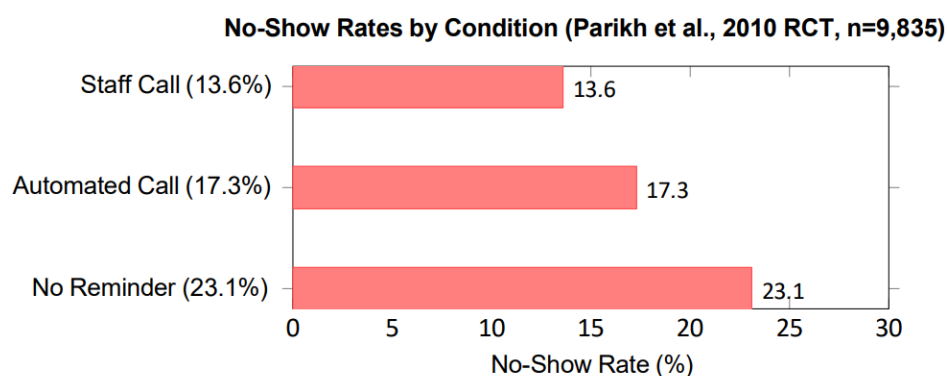


Figure 2.1: Overview of existing appointment scheduling challenges in service-based businesses

The review of relevant research and existing platforms shows that modern systems generally focus on online booking, availability management, and reminders. However, many of them are designed for general-purpose scheduling rather than for local service businesses that need staff coordination, service-specific booking rules, and business-level operational visibility.

2.2 Paper Reviews

A review of scheduling and service management literature indicates that appointment systems are most successful when they support real-time coordination, reduce administrative overhead, and improve customer communication. Earlier systems were mainly designed to automate calendars, but later solutions expanded to include reminders, confirmations, and availability checks.

Research in service operations suggests that booking systems are most effective when they help businesses manage time slots without creating scheduling conflicts. Frequent issues in conventional systems include conflict between staff schedules, poor visibility of occupancy, and limited support for service-specific duration and availability. These problems become more severe in businesses where different treatments require different durations and different staff skills.

In addition, the literature on customer behavior and service delivery emphasizes that missed appointments and late confirmations create financial losses and reduce customer trust. This is particularly relevant for service-based industries where customer retention depends on convenience, reliability, and professional communication. Therefore, the design of an effective booking system must consider both operational efficiency and customer experience.

Several studies in digital service technology also highlight that automation improves coordination when it is integrated into the booking cycle rather than used as an isolated add-on. For example, reminder messages, appointment confirmation, and real-time availability checks improve the efficiency of scheduling by reducing no-shows and helping customers take timely action. This is one of the major motivations behind developing a dedicated platform such as Slotify.

2.3 Comparison of Existing Platforms

Table 2.1 compares several existing scheduling platforms with the proposed Slotify system. The purpose is to identify the strengths of current tools and the missing features that are still not fully addressed for local service-based businesses.

From the comparison, it is clear that popular platforms provide useful features such as booking forms, reminders, and basic dashboards. However, they still tend to focus on general-purpose scheduling rather than the unique operational needs of small service businesses. Many tools do not provide strong support for separate staff scheduling rules, service-based availability, or business ownership control in one integrated system.

Slotify addresses these gaps by supporting business-specific workflows, multi-staff coordination, and a simplified booking experience tailored to service businesses. This makes it more suitable for local providers that require flexibility, affordability, and transparent operational control.

Table 2.1: Feature Comparison: Slotify vs. Existing Platforms

Feature	Calendly	Booksy	Acuity	SimplyBook	Slotify
Real-time slot display	✓	✓	✓	✓	✓
Automated email reminders	✓	✓	✓	✓	✓
Multi-staff support	✓	✓	✓	✓	✓
Analytics dashboard	Partial	✓	✓	✓	✓
Open-source / free	×	×	×	×	✓
Split-shift scheduling	×	Partial	Partial	✓	✓
AI-based slot suggestions	×	×	×	Partial	Planned
Payment integration	×	✓	✓	✓	Planned
Calendar sync	✓	Limited	✓	✓	✓
Self-hostable	×	×	×	×	✓

2.4 Reminder Effectiveness

Reminder systems are widely recognized as one of the most effective ways to reduce missed appointments and improve scheduling reliability. When a customer receives a confirmation and reminder before the appointment time, the chance of no-show decreases significantly. This effect is important in service businesses because even a small number of unfilled slots can cause lost revenue and poor customer flow.

The effectiveness of reminders is not only due to memory support but also because they create a stronger communication loop between the customer and the business. Automated reminders reduce reliance on manual follow-ups and ensure that customers are informed about their booking status. They also help businesses recover last-minute cancellations by allowing the time slot to be offered to other customers more quickly.

In this context, reminders are not an optional feature but a core part of the booking experience. A scheduling system that includes reminders can deliver better punctuality, lower cancellation risk, and stronger service continuity. This is one reason why Slotify emphasizes confirmation and reminder functionality as part of its core design.

2.5 Research Gap / What the Review Did Not Resolve

Although existing scheduling platforms offer useful features, they still do not completely solve the challenges faced by local service businesses. Many solutions are generic, while the actual needs of salons, clinics, therapy centers, and repair shops are often highly specific. The review also shows that no single platform fully combines flexible business management, multi-staff coordination, low-cost deployment, and customer-focused scheduling in a simplified form.

Another gap is the limited focus on service-specific business workflows. Many systems support appointment creation but do not adequately support staff availability rules, split scheduling, operational analytics, or business customization. In many cases, the solution is designed from the software side rather than from the actual daily challenges faced by business owners.

This review therefore highlights the need for a dedicated platform that is practical, scalable, and tailored to real service operations. Slotify is intended to fill this gap by combining online appointment booking, real-time availability checks, reminder communication, and business-oriented scheduling control in one unified system.

Chapter 3

Software Requirements Specification (SRS)

3.1 Purpose and Scope

The purpose of Slotify is to provide a digital appointment booking system for service-based businesses that need a reliable way to manage appointments, staff schedules, and customer communication. The system is designed to support online booking, walk-in scheduling, confirmation notifications, and business-level analytics while reducing manual coordination and scheduling errors.

The scope of the project includes customer booking, staff management, business administration, automated notifications, and real-time slot validation. The application is focused on service-oriented businesses such as salons, clinics, therapy centers, and repair shops, where appointment timing and service duration are essential to daily operations.

3.2 User Roles

The system supports multiple user roles with distinct responsibilities and access rights.

3.2.1 Customer

Customers can search for businesses, view available services, select time slots, and book appointments online. They also receive confirmations and reminders and may provide feedback after a completed service.

3.2.2 Business Admin

Business administrators manage working hours, service pricing, staff profiles, and business policies. They can monitor bookings, manage staff schedules, and review business performance.

3.2.3 Staff Member

Staff members can view their assigned appointments, confirm attendance, and complete service sessions. They are also responsible for maintaining an accurate schedule for the business.

3.2.4 Reception / Walk-In User

The receptionist or front desk user can create walk-in bookings for customers who do not book online. This role helps bridge the gap between digital and physical operations.

3.3 Functional Requirements

Slotify must provide functional capabilities that support appointment booking, schedule management, communication, and reporting.

- The system must allow a customer to browse a business and select a service.
- The system must display valid time slots based on business hours, staff availability, and existing bookings.
- The system must prevent double booking through validation and database-level constraints.
- The system must generate confirmation emails and reminder notifications.
- The system must support the creation of walk-in appointments from the admin panel.
- The system must allow staff and admins to manage bookings and status changes.
- The system must support review submission after a completed appointment.

3.4 Non-Functional Requirements

The application should be reliable, fast, secure, and easy to maintain. Performance is critical during peak booking periods when multiple customers may attempt to reserve the same available time slot. Security must be enforced through authentication, role-based access control, and tenant isolation. Maintainability also matters because the system may be extended with payment integration, analytics, and mobile support in future iterations.

3.5 Use Case Diagram

The use case model of Slotify includes actors such as the customer, business admin, staff member, and receptionist. The customer interacts with booking and review flows, while the admin and staff manage the business schedule and service execution. The receptionist supports walk-in requests and front desk operations. Together, these actors form the complete system workflow for appointment booking and service delivery.

3.6 Constraints

The system is constrained by several real-world requirements. Business schedules vary by day, staff member, and service category. The solution must therefore support flexible working hours and

Table 3.1: Non-functional requirements for Slotify

Requirement	Expectation	Design response
Performance	Slot availability must be returned quickly.	Indexed appointment queries and lightweight REST responses.
Reliability	A confirmed slot must not be offered twice.	Server-side validation and database-level conflict prevention.
Security	Users access only authorised information.	JWT authentication, role checks, and tenant-scoped data access.
Usability	Routine actions should be easy to complete.	Clear booking flow and role-specific dashboards.
Maintainability	New features should not disrupt the system.	Modular React components and controller–model–route backend structure.

service-specific durations. The system also needs to operate under a multi-business environment, where data isolation and secure access are essential. In addition, the project is limited by practical implementation constraints such as time, deployment environment, and the need to keep the system simple enough for business users to adopt without technical overhead.

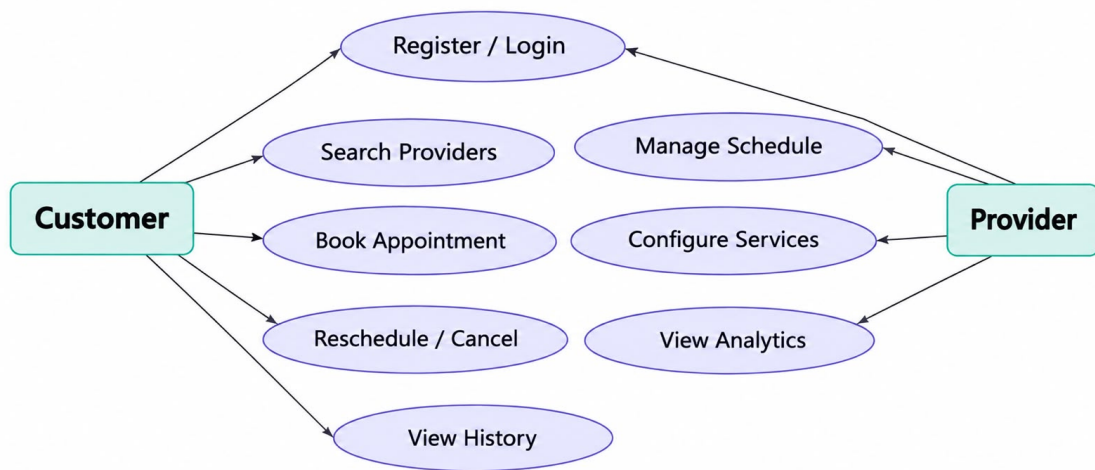


Figure 3.1: Use case diagram for Slotify user interactions and role-based workflow

Chapter 4

System Design

4.1 Architecture Overview

The architecture of Slotify is designed as a modular, multi-tier system that separates the user interface, application logic, and underlying data management. The frontend is responsible for user interaction, while the backend handles business logic, validation, notifications, and access control. A database layer stores all business, appointment, and user information in a structured and secure manner.

The design follows a multitiered approach so each component can be developed, maintained, and scaled independently. This structure also supports future extension, such as additional business modules or external integrations.

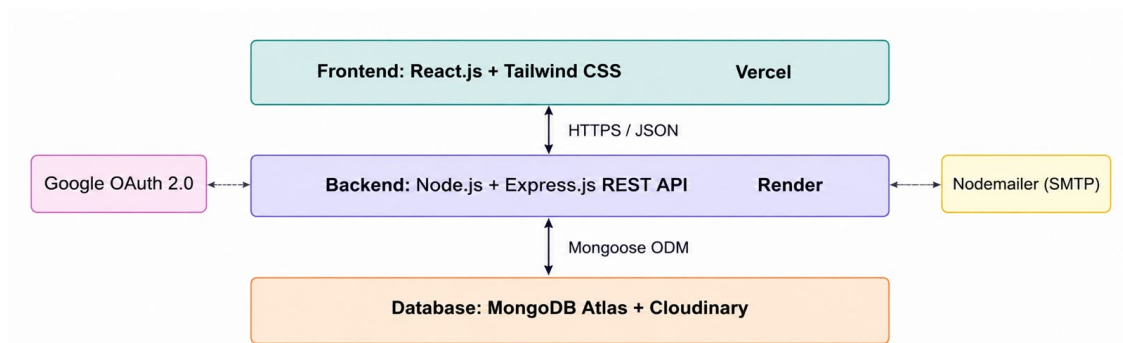


Figure 4.1: System architecture of the Slotify booking platform

4.2 Database Design

The database is designed to support multiple businesses, users, appointments, services, and staff records while maintaining data integrity. Business data is stored separately for each tenant, and appointment records are connected to the relevant business and staff member. This approach ensures that scheduling operations are isolated and accurate.

Key entities include users, businesses, services, staff, appointments, reviews, and notifications.

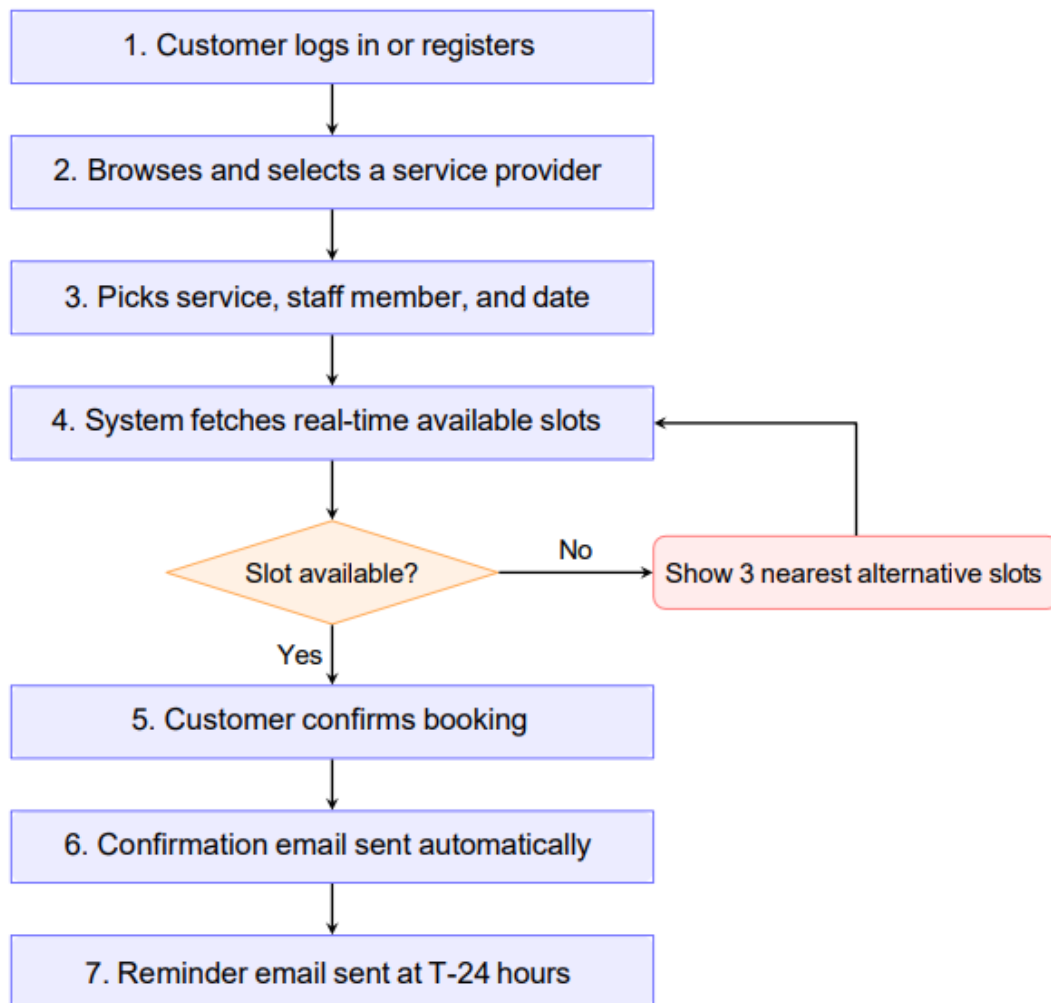


Figure 4.2: Database model and relationship flow for business, staff, and appointments

The schema is designed to support both routine operations and future analytics. Important fields such as appointment date, start time, service duration, and staff assignment are carefully captured to avoid conflicts and maintain historical accuracy.

4.3 API Design

The Slotify backend exposes a set of REST APIs to support authentication, business discovery, appointment creation, review handling, and analytics. These APIs are organized by domain and designed for clear separation of concerns. The appointment API handles availability checks and booking creation, while the auth API manages user login, registration, and token validation.

This API structure makes the system easier to extend and maintain. Future modules such as payment processing, analytics, or branch management can be integrated without disrupting existing endpoints.

Table 4.1: Key design modules and responsibilities

Module	Responsibility
Frontend Layer	Provides user-facing appointments, booking forms, dashboards, and business management screens
API Layer	Handles authentication, appointments, services, businesses, and reviews through structured endpoints
Business Logic Layer	Validates schedules, calculates slot availability, and enforces access rules
Database Layer	Stores users, appointments, business records, services, and staff availability data
Notification Service	Sends confirmations and reminder emails to customers and staff

4.4 Booking Workflow

The booking workflow begins when a customer selects a business and service. The system then checks real-time availability for the chosen staff member or next available option. If a valid slot is found, the booking is created with relevant service details and scheduling metadata.

Table 4.2: Booking process stages in Slotify

Stage	Description
Service Selection	The user browses businesses and selects the required service or treatment
Slot Check	The system validates available time slots based on staff, hours, and existing bookings
Booking Confirmation	A valid appointment is created and stored with required metadata
Notification Dispatch	Confirmation and reminder emails are sent to the customer
Schedule Update	The business schedule and admin records are updated for correct tracking

After booking, the system sends confirmation and reminder notifications and updates the business schedule. The workflow ensures that the slot is reserved only if all validations pass, reducing the risk of double booking and scheduling confusion.

4.5 Security Design

Security is an essential part of the Slotify design. User authentication is enforced through secure token-based mechanisms, and role-based access control prevents unauthorized actions. Business-

specific data is protected so one business cannot access another business's records.

Additional security considerations include validation of business ownership, prevention of unauthorized modifications, and controlled access to sensitive appointment data. These safeguards are important in a platform that handles customer and business information on a shared system.

4.6 UI Design Principles

The front-end design emphasizes simplicity, clarity, and speed. The user interface is designed to make it easy for customers to find services, check availability, and complete bookings without confusion. Business users also require dashboards that are intuitive and allow quick actions such as scheduling and managing appointments.

The user interface follows an accessible and consistent design approach to support usage across different devices and user roles. This helps create a smooth experience for customers, staff, and administrators while preserving usability across the application.

Chapter 5

Methodology

5.1 Development Approach

The development of Slotify followed a structured software engineering approach that combined requirement analysis, system design, implementation, and testing. The project was designed in phases so that each stage informed the next and helped maintain a clear path from concept to working system. This approach was important because the platform involves multiple user roles, booking workflows, and scheduling logic that must remain consistent across the system.

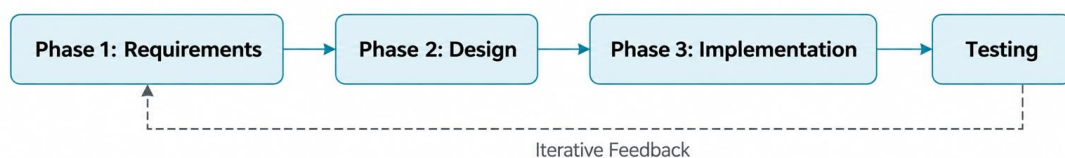


Figure 5.1: Methodology overview for the Slotify development lifecycle

5.2 Phase 1: Requirements Gathering

The first phase focused on identifying the core challenges faced by service-based businesses. This included understanding issues such as appointment conflicts, missed bookings, manual scheduling, and lack of business visibility. Input from the problem statement and literature review helped define the critical functional and non-functional requirements for the project.

5.3 Phase 2: System Design

Once the requirements were clear, the architecture and data model were designed to support multi-user scheduling. The design included a modular frontend, backend API layer, and database schema intended to handle business operations, service scheduling, and role-based access. The design phase also defined the flow of booking creation, confirmation, and reminder communication.

5.4 Phase 3: Implementation

The implementation phase involved building the backend logic, creating the frontend interfaces, and integrating the scheduling components. Core functionality such as slot generation, appointment validation, authentication, and notification services were implemented during this stage. The project was developed in a way that allowed real system operations to be tested with realistic booking scenarios.

5.5 Phase 4: Testing

Testing was carried out to validate that the application could create appointments correctly, prevent conflicts, and respond reliably under different conditions. This included checking the booking flow, verifying role access, and testing the reliability of slot calculation. The testing phase also helped identify issues related to concurrency and database constraints.

5.6 Tools and Environment

The project was developed using a modern web development stack to support scalability and maintainability. The frontend was built using React, while the backend used Node.js and Express. MongoDB was used for database storage, and email-based notifications were integrated through a transactional service. These tools were selected to support both the scheduling logic and the overall user experience.

Table 5.1: Tools used during development

Category	Tool	Purpose
Version control	Git and GitHub	Source control and team collaboration
IDE	Visual Studio Code	Primary development environment
API testing	Postman	Endpoint testing and collection management
UI prototyping	Figma	Wireframes and layout planning
Database GUI	MongoDB Compass	Schema inspection and visual debugging
Frontend deployment	Vercel	Continuous deployment from GitHub
Backend deployment	Render	Node.js cloud hosting
Database hosting	MongoDB Atlas	Managed cloud database

Table 5.2: Development phases and expected outputs

Phase	Outcome
Requirements Gathering	Problem definition, business needs, and functional requirements identified
System Design	Architecture, database model, and workflow structure planned
Implementation	Frontend, backend, and scheduling logic developed
Testing	Appointment logic verified for reliability, conflict prevention, and correctness

Chapter 6

Implementation

6.1 Backend Implementation

The backend of Slotify is built using Node.js and Express.js. It manages authentication, appointment processing, service queries, and business logic. A structured routing and controller pattern is used to keep the application organized and easier to maintain as the project grows.

6.1.1 Project Structure

The backend project is organized into modular folders such as controllers, models, middleware, routes, and utilities. This separation allows the system to manage business logic, validation, and database access more efficiently. A clear project structure is essential in a platform that supports multiple user roles and different services.

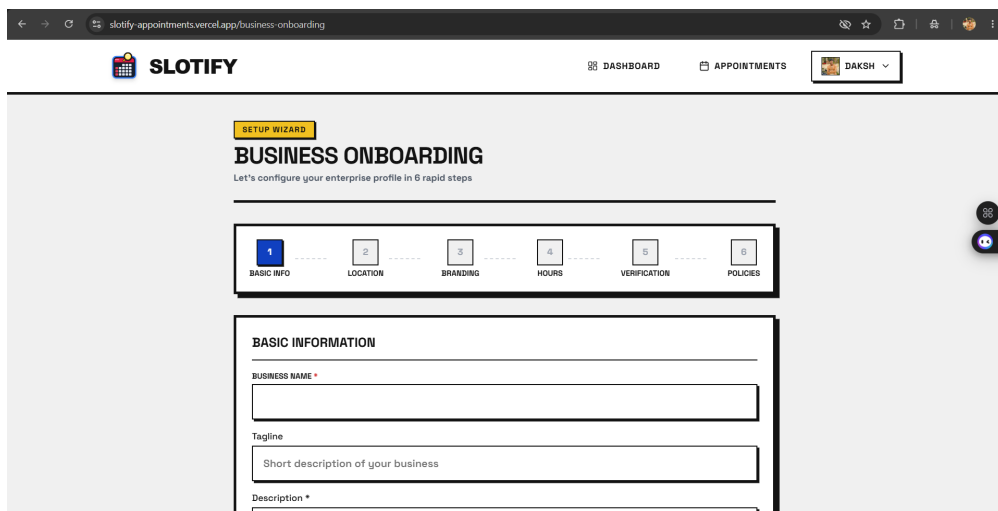
The screenshot displays the 'BUSINESS ONBOARDING' setup wizard in the Slotify application. The interface includes a top navigation bar with the Slotify logo, a 'DASHBOARD' link, an 'APPOINTMENTS' link, and a user profile for 'DAKSH'. The main content area features a progress bar with six steps: 1. BASIC INFO (active), 2. LOCATION, 3. BRANDING, 4. HOURS, 5. VERIFICATION, and 6. POLICIES. Below the progress bar, the 'BASIC INFORMATION' section contains three input fields: 'BUSINESS NAME' (with a red asterisk indicating a required field), 'Tagline', and 'Description' (also with a red asterisk). The browser's address bar shows the URL 'slotify-appointments.vercel.app/business-onboarding'.

Figure 6.1: Business configuration and setup screen

6.1.2 Slot Availability Logic

The slot availability engine checks for business working hours, staff schedules, and existing bookings before rendering available times. It uses rules such as service duration, business policies, and staff

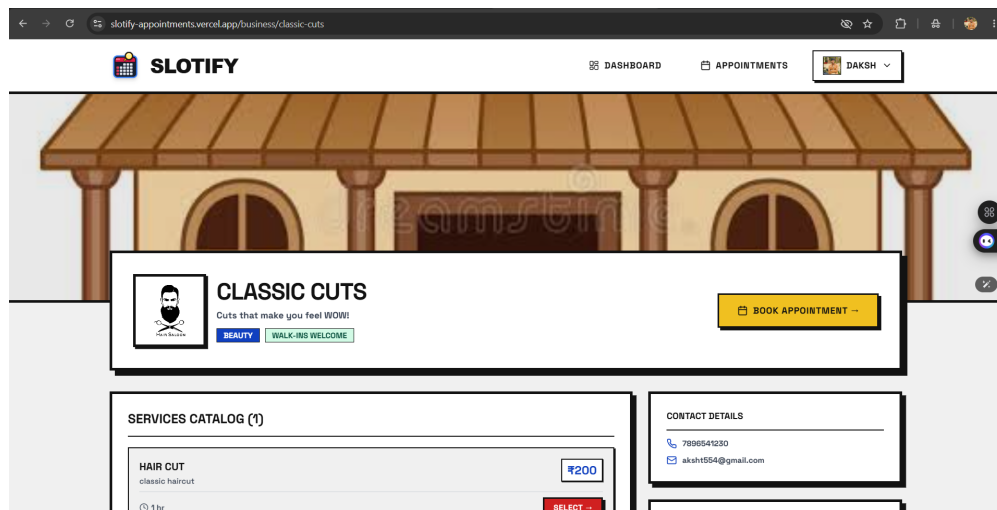


Figure 6.2: Business profile and service discovery interface

availability to determine valid appointment windows. This logic prevents conflicts and maintains the reliability of the booking system.

6.1.3 Authentication Middleware

Authentication middleware verifies user identity before allowing protected API access. It ensures that users are logged in and that only authorized users can view or modify relevant business data. This is important for preventing unauthorized actions and enforcing role-based access.

6.1.4 Email Notification Service

The system includes an email notification service to send confirmation messages and reminders to customers. These notifications help reduce missed appointments and improve communication between the business and the customer. The service is integrated into the booking and reminder flow to keep the scheduling process reliable.

6.2 Frontend Implementation

The frontend is built using React, which provides a responsive and interactive interface for customers, admins, and staff. It is designed to present a clear booking flow, show available time slots, and manage user actions across different roles.

6.2.1 Project Structure

The frontend is organized into reusable components, pages, routes, and context-based state management. This structure improves maintainability and allows the UI to scale as the platform evolves. It also keeps the login, dashboard, and business booking flows separated into well-defined modules.

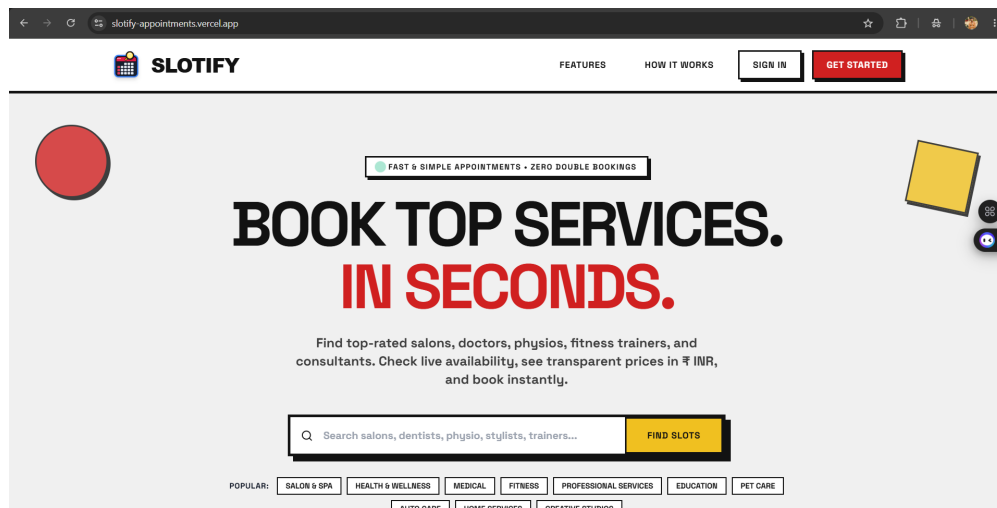


Figure 6.3: Landing page of the Slotify platform

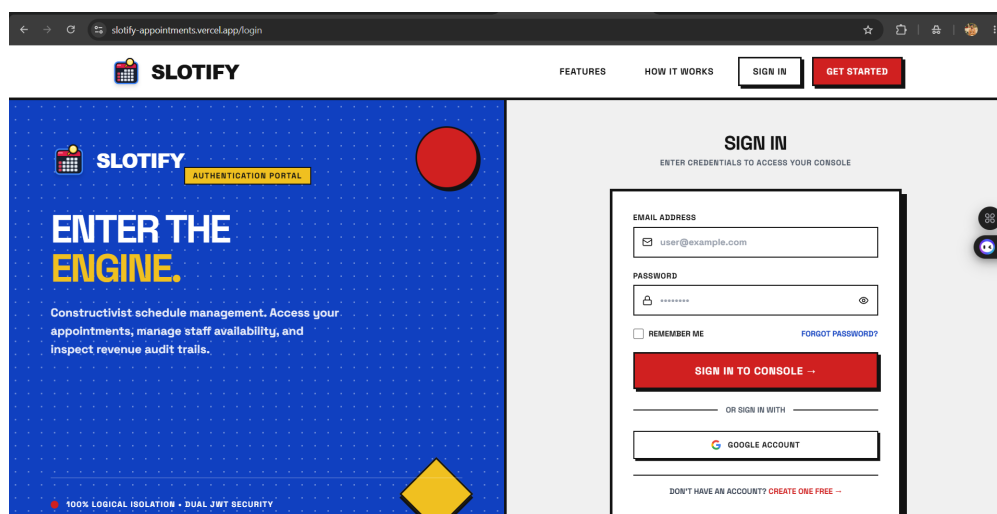


Figure 6.4: Login interface for the user authentication flow

6.2.2 Real-Time Slot Display

The booking interface updates available time slots dynamically based on current data and business rules. Customers can see only valid appointment times, which improves the user experience and prevents the confusion associated with manual scheduling. The real-time slot display is a core part of the platform's operational value.

6.2.3 State Management and Prop Drilling Refactor

As the application grew, state management became more complex. The project addressed this by reducing unnecessary prop drilling and centralizing shared state where necessary. This made the application more maintainable and improved the consistency of booking, authentication, and dashboard data across components.

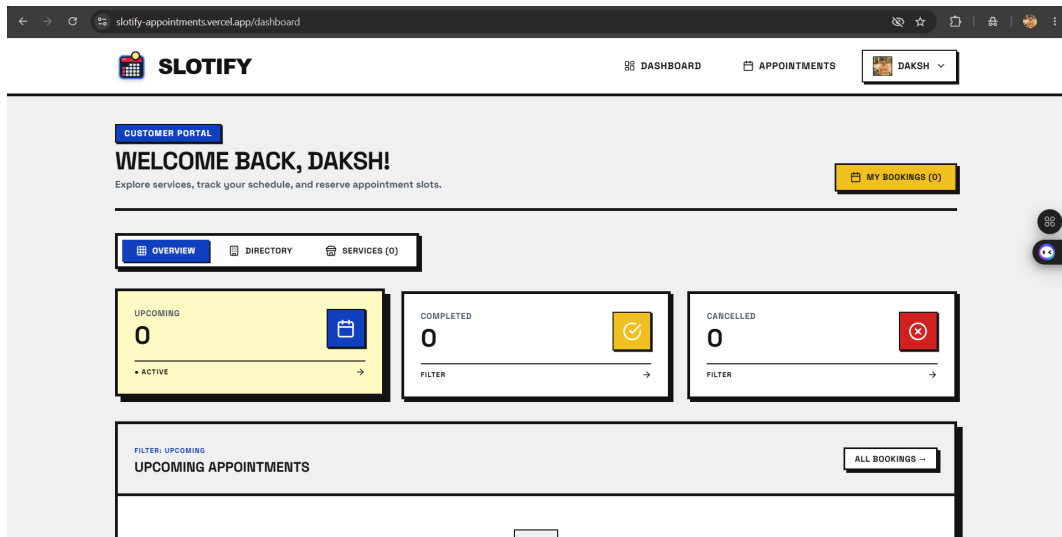


Figure 6.5: Admin dashboard showing bookings, activities, and business overview

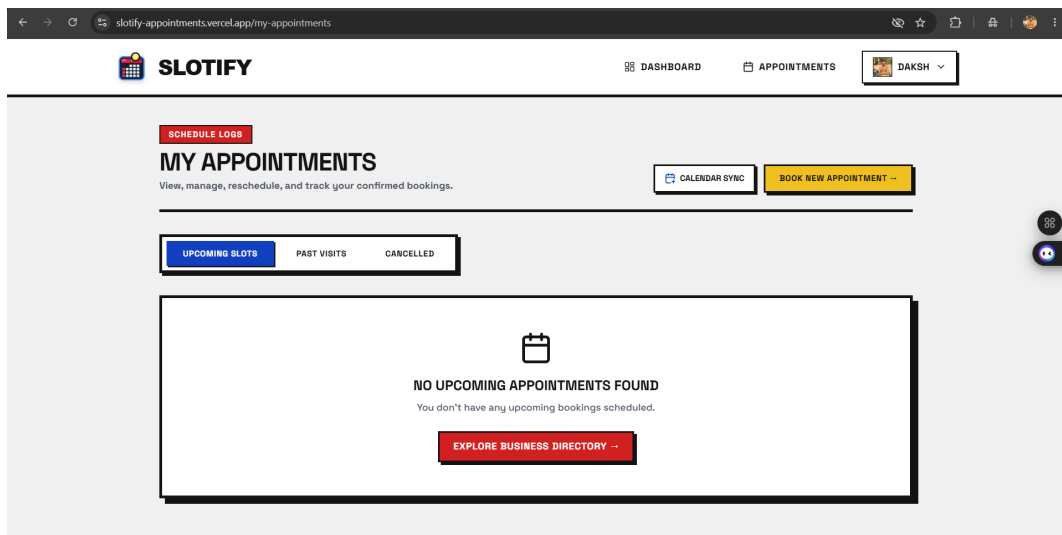


Figure 6.6: Appointment list and scheduling records

6.3 Implementation Status

The application has implemented the core features needed for a functional booking system, including user authentication, business discovery, service booking, reminders, and scheduling logic. The project is now in a working state and demonstrates the viability of the solution for small service-based businesses. Further enhancements such as payments, AI recommendation, and mobile support can be added as future extensions.

Table 6.1: Feature implementation status

Feature	Status	Phase
User registration and login	Done	Semester 6
Real-time slot booking	Done	Semester 6
Email confirmation and reminder	Done	Semester 6
Provider dashboard and schedule	Done	Semester 6
Business analytics	Partial	Semester 6
Calendar synchronisation	In progress	Semester 6/7
Payment integration, mobile app, and AI recommendations	Deferred	Semester 7

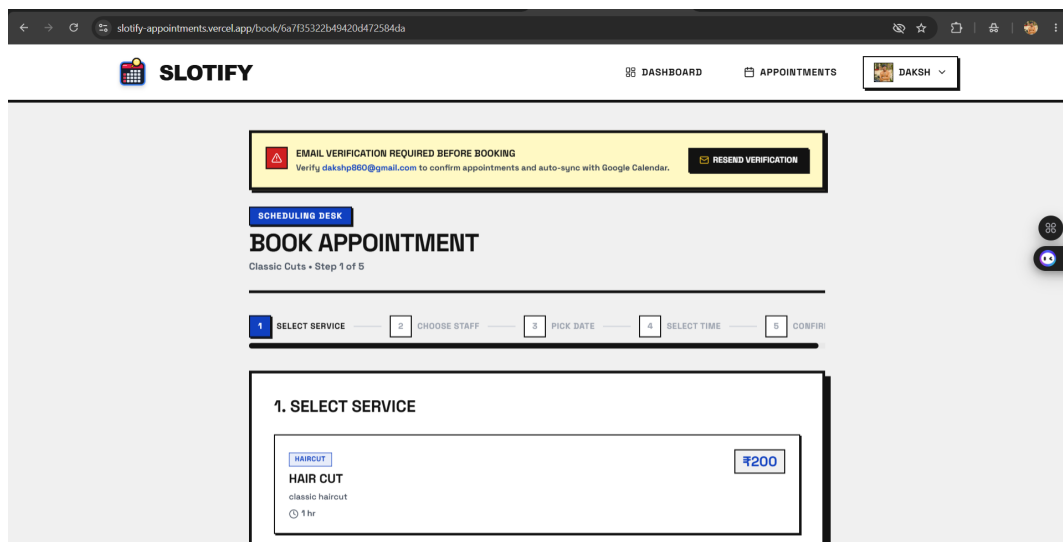


Figure 6.7: Appointment lifecycle flow from booking to service completion

Chapter 7

Conclusion

7.1 Summary of Work Done

The Slotify project was developed to solve the recurring scheduling and coordination issues faced by service-based businesses. The platform brings together appointment booking, staff management, business scheduling, customer communication, and review handling into a unified system. By addressing the common problems of missed appointments, manual coordination, and scheduling conflicts, the project demonstrates a practical solution for modern service businesses.

7.2 Objectives Met

The project successfully met its major objectives by enabling customers to search for services and book appointments online, helping businesses manage staff and working schedules, and supporting automated reminders to improve booking reliability. It also introduced role-based access and a structured application flow that makes the system suitable for real operational use.

Table 7.1: Objective completion status

No.	Objective	Status
1	Customer-facing booking with real-time slot display	Completed
2	Provider dashboard with schedule management and double-booking prevention	Completed
3	Automated email confirmation and 24-hour reminder	Completed
4	Calendar synchronisation and cloud deployment	Partial / Completed
5	Business analytics module	Partially done

7.3 Challenges Encountered

Several technical and practical challenges were encountered during development, especially in the areas of slot validation, concurrency handling, and user-role management. Synchronizing booking

data across multiple users while maintaining consistency was one of the most significant technical challenges. In addition, the system had to balance usability and complexity so that it remained understandable for business users while supporting backend reliability.

7.4 Key Takeaways

The project highlighted the importance of designing scheduling systems around real business problems rather than generic software features. A strong appointment system must balance customer convenience, business efficiency, and technical reliability. Slotify demonstrates that this balance is achievable through careful design, modular development, and strong validation of booking logic.

Chapter 8

Future Work

8.1 What Comes Next

Slotify has established a strong foundation for digital appointment scheduling, and the next stage of development can focus on expanding the platform's capabilities to support larger user bases and more advanced business workflows. The future direction of the project is guided by customer needs, business scalability, and the growing demand for smarter scheduling systems.

8.2 Payment Integration

One of the most important future enhancements is payment integration. By supporting digital payments, the system can allow deposits, service charges, and cancellation policies to be handled directly during the booking process. This will strengthen the platform's business usability and improve financial management for service providers.

8.3 Mobile Application

A mobile application would make the system more convenient for both customers and businesses. Customers could book appointments from a mobile device, receive reminders, and manage ongoing schedules more easily. Business owners and staff would also benefit from a mobile dashboard for appointment updates and service tracking.

8.4 AI-Based Slot Recommendations

Artificial intelligence can be incorporated to suggest optimal appointment slots based on historical booking patterns, demand trends, and staff availability. This would reduce idle time, improve resource utilization, and help businesses manage high-traffic periods more efficiently.

8.5 Advanced Analytics

Future versions of Slotify can include more detailed analytics for business performance. These could include peak-hour analysis, service popularity trends, customer retention insights, and staff productivity metrics. Advanced analytics would help business owners make more informed decisions about scheduling and operations.

8.6 Multi-Language Support

Multi-language support would improve accessibility for businesses that serve diverse customer groups. Supporting more languages would make the platform more inclusive and easier to adopt across different regions and communities.

8.7 Multi-Branch Business Support

A major future extension is support for multi-branch businesses. This would allow a single business owner to manage several locations, staff groups, and appointment flows from one platform. It would be especially useful for chains, clinics, and service networks with multiple branches.

Chapter 9

Research Paper / Survey Paper and Plagiarism Report

This chapter contains the project research paper and the plagiarism report in their original submitted form. They are included without modification so that their content, pagination, and institutional formatting remain intact.

9.1 Research Paper

Slotify: A Smart Web-Based Online Appointment Scheduling Platform for Service-Based Businesses

Aksh Thakkar

Project Lead & Full-Stack Dev.

Dept. of CSE

Parul Institute of Engineering
and Technology, Gujarat, India

2403031057002@paruluniversity.ac.in

Om Maurya

Frontend Developer (UI/UX)

Dept. of CSE

Parul Institute of Engineering
and Technology, Gujarat, India

2403031057048@paruluniversity.ac.in

Daksh Patel

Backend Developer

Dept. of CSE

Parul Institute of Engineering
and Technology, Gujarat, India

2403031057035@paruluniversity.ac.in

Jainil Patel

Database & Integration Eng.

Dept. of CSE

Parul Institute of Engineering
and Technology, Gujarat, India

2403031057071@paruluniversity.ac.in

Dr. Sneha Soni

Associate Professor (CSE)

Dept. of CSE

Parul Institute of Engineering
and Technology, Gujarat, India

sneha.soni37803@paruluniversity.ac.in

Abstract—Small and medium service-oriented enterprises – clinics, grooming studios, coaching centres, and consultancy firms – still coordinate bookings largely by phone and paper register, a habit that routinely produces double-booked slots, forgotten appointments, and staff time that nobody bothers to account for. This paper presents Slotify, a browser-accessible, multi-tenant booking platform built on the MERN stack (MongoDB, Express.js, React.js, Node.js) that lets independent businesses onboard under their own public storefront while sharing a single application instance. The system’s central engineering contribution is a slot-availability engine that resolves staff working hours, breaks, holidays, buffer times, and advance-booking windows into a live grid of open slots, backed by a database-enforced unique constraint that makes a race-condition double booking structurally impossible rather than merely unlikely in practice. We document the architecture, the booking and reschedule algorithms in pseudocode, and a feature-by-feature comparison against three commercial platforms actually in use today – Calendly, Booksy, and Fresha – as well as two previously published academic prototypes. Unlike prior systems, which each cover a fragment of the scheduling lifecycle, Slotify combines tenant-scoped multi-staff scheduling, audited reschedule history, and a public discovery layer without per-seat licensing or marketplace commission. We close by discussing, with reference to existing clinical and operational literature rather than unverified benchmarks of our own, where the architecture is expected to help and where its current single-location, English-only scope still falls short of production-grade commercial offerings.

Keywords—Online Booking System, MERN Stack, Multi-Tenant Architecture, Slot Management, Atomic Concurrency Control, JSON Web Token, Automated Reminders, MongoDB Atlas, Appointment Audit Trail

I. INTRODUCTION

A hair salon owner in a mid-sized Indian city still runs her appointment book the way her mother did: a diary by the register, a phone that rings during the one hour she has free to actually cut hair, and a mental note of which regulars tend to no-show on Mondays. This is not a failure

of ambition – it is what happens when the tools available to a five-person clinic or salon are either built for a single professional’s personal calendar or priced and structured for a national retail chain. Growing consumer expectations for on-demand digital services have exposed exactly this gap. Clinics, beauty parlours, tutoring institutes, and fitness studios still depend heavily on receptionist-managed phone queues and handwritten registers, an approach that scales poorly and introduces avoidable operational friction [6]. When a booking record lives only in a notebook, one clerical slip cascades into overlapping reservations, wasted service capacity, and a client who does not come back [2].

The economic argument for fixing this is not speculative. Multiple health-system studies report that reminder-equipped digital platforms measurably cut no-show rates, though – as we discuss in Section II, and contrary to how this finding is sometimes summarised – the size of the effect depends heavily on the care setting, not just the presence of a reminder [7]. Primary-care research shows that structured online booking meaningfully compresses the administrative burden on front-desk teams [3], [4], and a 2024 industry survey found that businesses without any centralised booking tool report more staff overtime and more revenue that simply evaporates unbilled [6].

This paper introduces Slotify: a browser-accessible, multi-tenant appointment platform built on the MERN stack [19], [20]. What actually distinguishes it from a personal scheduling link is not any single feature but the fact that a business – with its own staff roster, working hours, and service catalogue – is a first-class object in the data model, and that a customer can discover that business through a shared public directory rather than needing its URL in advance.

The engineering core of what follows is really one contribution wearing two hats: a three-tier architecture and slot-resolution algorithm, expressed as pseudocode, that turns staff working hours, breaks, holidays, and buffer rules into a

live grid of bookable slots, paired with a database-enforced concurrency scheme – a finite-state model plus an atomic reservation algorithm – that removes double-booking as a possible outcome rather than merely reducing its probability. We separate the two only because the second doesn't make sense without the first: there is no point guaranteeing atomic writes against a slot grid that was computed wrong to begin with.

Beyond that, we contribute a grounded, source-verified comparison of Slotify against three commercial platforms in active use (Calendly, Booksy, Fresha) and two previously published academic prototypes, built from each vendor's own documentation rather than secondary marketing copy. And we say plainly, in Section IX, what we have not done: Slotify has not been load-tested or put through a formal security audit, and no simulated number stands in for that missing work.

II. RELATED WORK

A. Evolution of Appointment Booking Platforms

Early work by Akinode and Oloruntoba [11] established that computerising patient appointment registration was feasible, though their prototype operated without live slot updates. Atherton et al. [3] found that patients using portal-based booking reported higher satisfaction than those relying on telephone channels, and a JMIR evidence synthesis [5] catalogued recurring design gaps across eighteen published web-based medical scheduling systems – live availability, automatic reminders, and calendar linkage came up again and again as the features users wanted and did not have. Those three gaps shaped Slotify's feature set directly.

Srivastava et al. [1] linked scheduling friction to post-visit dissatisfaction, and Shayan et al. [8] surfaced three satisfaction predictors from clinic stakeholders – responsiveness, notification reliability, and role-appropriate interfaces – that shaped Slotify's UX priorities more than any single design guideline did. Kaluza et al. [9] gave a formal treatment of how visible occupancy in a booking queue can be used to signal service quality, an idea that motivated the analytics view in Slotify's provider dashboard even though we have not yet built out the predictive layer they describe.

The most carefully controlled evidence on reminders and no-shows, however, complicates the simple story. Betancor et al. [7] compared online versus telephone appointment scheduling across both a private ophthalmology practice and a university hospital eye clinic. In the private practice, the no-show rate fell from 5.9% to 1.8% after online scheduling was introduced – a genuinely large relative improvement. In the hospital setting, however, no-shows did not improve at all; they rose slightly, from 11.2% to 14.3%. The authors' own reading is that patient population and appointment lead time, not the booking channel itself, drove the difference. We take this as a caution rather than an inconvenience: a MERN-based booking layer like Slotify is a necessary condition for the kind of improvement seen in the private-practice arm of that study, but it is evidently not sufficient on its own, and we do not claim otherwise.

B. Domain-Specific Web Booking Implementations

Wankhade et al. [15] built a functional appointment tool aimed at connecting patients, doctors, and pharmacy vendors; the published description does not detail live slot synchronisation across concurrent users, which is precisely the gap a document-oriented, event-driven stack is suited to close. Chauhan et al. [16] paired a physician directory with a Django-based booking back end, validating demand for a combined discovery-and-booking journey – the same journey Slotify's public business-discovery layer targets, but for salons, tutors, and consultants rather than doctors specifically. Additional published prototypes [12]–[14], [17], [18] reinforce that the demand for digital scheduling is sector-wide, while also showing that no single prior academic tool combines multi-tenant business accounts, staff-level scheduling, and a public marketplace in one system.

C. MERN as an Application Platform

Baiskar et al. [19] found MERN particularly well suited to applications with frequent, event-driven data mutations – a description that matches a live booking system closely, since every slot query and every reservation attempt is exactly that kind of mutation. A comparative study of the stack [20] noted that MongoDB's schema-flexible document model absorbs evolving service and slot structures with less migration overhead than a rigid relational schema would require, which matched our own experience adding fields like `bufferTime` and `unavailableDates` well after the initial schema was designed.

D. Stateless Authentication and Reminder-Driven Attendance

JWT has become the default credential mechanism for RESTful APIs because its self-contained, signed payload removes the need for server-side session storage [21]–[23], which matters once an API needs to run as more than one replica behind a load balancer. Pairing JWT with Google OAuth lets a first-time customer book an appointment without first inventing a password, removing what is often the single largest drop-off point in a booking funnel.

On reminders specifically: Ginat and Kenniff [24] showed a multi-modal reminder system at a radiology unit produced a statistically significant drop in unfilled scan slots. Parikh et al. [25] identified email as a high-impact channel for reducing no-shows among working-age patients. Tarabichi et al. [26] ran a randomised quality-improvement trial layering predictive, risk-driven telephone outreach on top of standard automated reminders in a safety-net primary care clinic; their central finding was about equity – whether the intervention narrowed or widened no-show disparities across racial and ethnic groups – rather than a single headline percentage, and we cite it here for that finding rather than for a number that would misrepresent the study's actual design. Two systematic reviews together cover more than thirty independent reminder trials and reach a consistent, if modest, conclusion: automated reminders help, but they are not a complete fix on their own [27], [28]. That is the honest justification for Slotify's cron-scheduled 24-hour reminder pipeline – a defensible design choice grounded in a real and repeated finding, not a guarantee.

E. Identified Research Gap

Across this literature, no single JavaScript-native platform combines multi-tenant business accounts, live staff-level slot resolution, database-enforced conflict prevention, and a public discovery directory in one open, self-hostable codebase. Commercial platforms that come close – surveyed in Section V – solve this by charging per seat or by taking a commission on marketplace-driven bookings, which is a reasonable business model but a real cost for a business running on thin margins. Slotify is offered as a concrete, if less commercially mature, alternative to that trade-off.

III. PROBLEM STATEMENT

Three related failure modes recur across service micro-enterprises that still run scheduling manually.

Unstructured booking channels and chronic non-attendance turn out to be the same problem seen from two angles. Coordinating reservations across phone calls, WhatsApp threads, and a paper register introduces inconsistency at every touchpoint, and staff must reconcile overlapping requests without any algorithmic guard against double allocation [6], [11]. The same absence of a system-of-record is what makes a booking, made days in advance with no subsequent touchpoint, easy for a customer to forget or deprioritise. Reported non-attendance rates in purely manual environments commonly sit between 15% and 30% [25], [27], and that spread is wide precisely because it depends on whether anything nudges the client between booking and appointment day – which, in a manual system, nothing does. Outside business hours, no booking mechanism exists at all, quietly capping demand the business never even sees.

Fragmented operational visibility is the third and more diffuse problem. A business without a centralised booking repository cannot measure slot utilisation, cannot track cancellation patterns, and cannot forecast a busy week [9], [10]. Growth decisions get made on gut feeling, and syncing the schedule with a staff member’s personal calendar means manually re-typing it.

IV. SYSTEM ARCHITECTURE

A. Architectural Overview

Slotify uses a conventional three-tier decomposition – a React.js client, an Express.js REST layer, and a MongoDB Atlas data store – chosen specifically so that any tier can be scaled or replaced without touching the other two. Fig. 1 shows the logic tier’s internal module boundaries: authentication, the scheduling engine, the notification dispatcher, and the analytics aggregator sit behind a shared API router, each with its own area of responsibility and its own collection(s) to read and write.

Tenant isolation is logical, not physical: every business, service, staff, and appointment document carries a `businessId` field, and every query the application issues is scoped by it. This was a deliberate trade-off. Spinning up a database per tenant would have made the isolation guarantee stronger and the operational story much worse – a hundred small salons would mean a hundred databases to back up, migrate, and monitor. A single shared collection set with compound

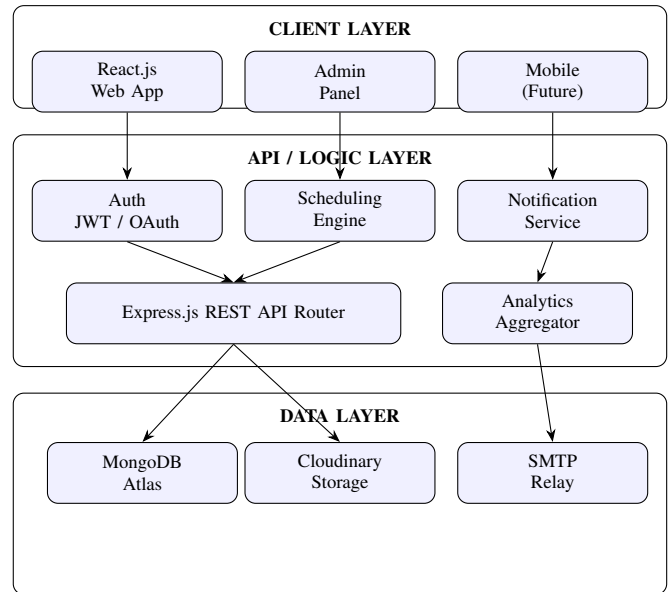


Fig. 1. Three-tier architecture of Slotify: client, API/logic, and data layers, with the scheduling engine and notification dispatcher as independent modules behind a shared router.

indexes on `businessId` keeps operations simple at the cost of relying on application-level query discipline rather than the database engine itself to enforce the boundary. For a platform at this stage, we judged that trade worth making; it would need revisiting well before Slotify tried to serve regulated data like health records at scale.

V. COMPARATIVE ANALYSIS WITH EXISTING PLATFORMS

A fair comparison has to start from what these platforms actually publish about themselves, not from how they are usually summarised in blog posts, so the entries below are drawn from each vendor’s own pricing and feature pages as of mid-2026, plus the two academic prototypes already discussed in Section II.

Calendly [31] is, at its core, a personal and team meeting scheduler. It checks one or more of a user’s own connected calendars (Google, Outlook, iCloud) for conflicts and writes new bookings into them; multi-calendar conflict checking, payment collection via Stripe/PayPal, and round-robin team distribution are all gated behind its Standard and Teams tiers, priced per user per month. There is no concept of a business entity with its own staff roster and service catalogue, and no public discovery marketplace – a customer needs the specific booking link, not a directory search.

Booksy [32] is much closer in spirit to Slotify: it is explicitly multi-tenant, gives each business a public storefront, and operates a consumer-facing marketplace where new clients discover salons and barbershops by category and location. It bundles staff scheduling, point-of-sale, and marketing tools into a single per-location subscription (advertised from \$29.99/month) with additional per-staff charges, and it markets a no-show reduction of up to 25% through deposits and reminders – a vendor claim we report as such, not as an independently verified figure.

Fresha [33] takes a different commercial approach: the core subscription is free, and Fresha instead charges a one-time 20% commission on new clients acquired through its own marketplace, plus standard payment-processing fees. Feature-wise it covers staff shift scheduling, commission tracking, POS, and automated SMS/email reminders, aimed squarely at the beauty, wellness, and healthcare verticals.

Table I summarises the comparison on criteria that matter specifically for a multi-staff service business, rather than a generic feature checklist. An entry is marked *Unconfirmed* where the vendor's own documentation did not settle the question either way.

The pattern across the three commercial products is consistent: whichever one you pick, you are trading a percentage of revenue or a per-seat fee for genuine production maturity – payment processing, mobile apps, years of edge-case handling. Slotify's contribution is not that it does any one of these things better; it does not yet do most of them at all. What it offers is the specific combination of multi-tenant scheduling logic and a database-level correctness guarantee against double booking, built on a stack a small development team can read, modify, and deploy without a recurring commission on every new client. That is a narrower claim than a feature-count table can fully convey, and it is the honest one.

VI. BOOKING AND CONCURRENCY ALGORITHMS

The scheduling engine's correctness rests on two pieces of logic: turning a day's working hours into a list of candidate slots, and making the act of claiming one of those slots atomic at the database level so that two customers racing for the same slot cannot both win.

A. Slot Resolution

Algorithm 1 mirrors the logic implemented in `availabilityCalculator.js`. It short-circuits early on holidays and closed days, then for each eligible staff member walks through the working-day window in increments of the requested service's duration, discarding any candidate slot that falls inside a break, inside the minimum-advance window, beyond the maximum-advance window, or inside an existing appointment's buffer-inclusive span.

Two things about this algorithm are worth being direct about. First, it runs identically on the server for every booking attempt – the client-side calendar view is a convenience, not a source of truth, precisely because a client-supplied timestamp cannot be trusted to reflect what is still actually open by the time the request lands. Second, the deduplication priority in line 22 was not obvious in the first draft of the design: an early version simply took whichever staff record was queried first, which meant a slot could show as BOOKED to a customer even when a different staff member covering the same service was, in fact, free. Preferring AVAILABLE across all matching staff records fixed that, at the cost of the slightly more complex merge step shown above.

B. Atomic Reservation and Conflict Resolution

Fig. 2 models a slot's lifecycle as a four-state automaton. A viewer browsing the calendar places a soft, non-blocking

Algorithm 1 ComputeAvailableSlots

Require: business B , service S , date d , optional staff filter F
Ensure: ordered list of slot statuses for date d

```

1: if  $d \in B.\text{holidays}$  or not  $B.\text{workingHours}[d].\text{isOpen}$  then
2:   return  $\emptyset$  ▷ business closed that day
3: end if
4:  $\text{staffList} \leftarrow$  active staff assigned to  $S$  (filtered by  $F$  if given)
5:  $\text{slots} \leftarrow \emptyset$ 
6: for each staff  $st$  in  $\text{staffList}$  do
7:   if  $d \in st.\text{unavailableDates}$  then continue
8:   end if
9:    $w \leftarrow st.\text{workingHours}[d]$  or  $B.\text{workingHours}[d]$ 
10:   $t \leftarrow w.\text{start}$ 
11:  while  $t + S.\text{duration} \leq w.\text{end}$  do
12:     $\text{cand} \leftarrow [t, t + S.\text{duration})$ 
13:    if  $\text{cand}$  overlaps  $st.\text{breaks}$  then
14:      mark  $\text{cand}$  as UNAVAILABLE
15:    else if  $t < \text{now} + B.\text{minAdvance}$  then
16:      mark  $\text{cand}$  as TOO SOON
17:    else if  $t > \text{now} + B.\text{maxAdvance}$  then
18:      mark  $\text{cand}$  as TOO FAR
19:    else if  $\text{cand}$  overlaps any scheduled appt for  $st$  (incl. buffer) then
20:      mark  $\text{cand}$  as BOOKED
21:    else
22:      mark  $\text{cand}$  as AVAILABLE
23:    end if
24:     $\text{slots} \leftarrow \text{slots} \cup \{(t, st, \text{status})\}$ 
25:     $t \leftarrow t + S.\text{duration}$ 
26:  end while
27: end for
28: deduplicate overlapping  $(t, \cdot)$  entries across staff, preferring AVAILABLE  $\succ$  TOO SOON  $\succ$  TOO FAR  $\succ$  BOOKED
29: return  $\text{slots}$ 

```

VIEWED marker; confirming a booking attempts to promote the slot directly to RESERVED via a single conditional write.

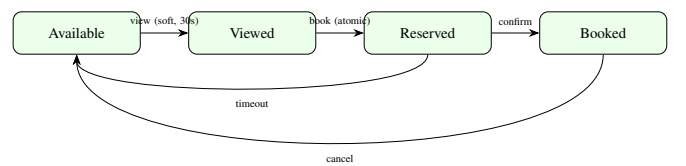


Fig. 2. Slot lifecycle state machine. The *book* transition is the single atomic write that either succeeds outright or fails cleanly, with no partial state in between. The *timeout* and *cancel* return paths are drawn at different depths so they do not cross.

Algorithm 2 shows the reservation attempt itself. The critical property is that step 2 is a single conditional database operation (a `findOneAndUpdate`-style compare-and-set backed by the unique partial index on $\{\text{businessId}, \text{staffId}, \text{appointmentDate}, \text{startTime}\}$ filtered to $\text{status} = \text{"scheduled"}$), not a read followed by a separate write. Two customers racing for the same slot both reach step 2, but the database engine, not application code, decides which write lands first and rejects the other – which is the only place a guarantee like this can actually live, since two Node.js request handlers running concurrently have no shared memory

TABLE I. FEATURE COMPARISON: SLOTIFY AGAINST COMMERCIAL PLATFORMS AND PRIOR ACADEMIC PROTOTYPES

Capability	Slotify	Calendly [31]	Booksy [32]	Fresha [33]	Academic prototypes [15], [16]
Multi-tenant business accounts	Yes	No	Yes	Yes	No
Public discovery / marketplace	Yes (own directory)	No	Yes	Yes	No
DB-enforced double-booking prevention	Yes (unique partial index)	Partial (relies on external calendar sync)	Yes	Yes	Unconfirmed
Staff-level working hours & buffers	Yes	Partial (per-user only)	Yes	Yes	Unconfirmed
Reschedule limits with audit trail	Yes	No	Unconfirmed	Partial	No
JWT + Google OAuth	Yes	Google Calendar-based	Unconfirmed	Unconfirmed	Partial [16]
Automated email reminders	Yes	Yes	Yes	Yes	Unconfirmed
Self-hostable / no per-seat fee	Yes	No (per-user SaaS)	No (per-location + fees)	No (marketplace commission)	Yes (published prototype)
Production hardening & scale	Not demonstrated	yet Extensive	Extensive	Extensive	Not demonstrated

to coordinate through.

Algorithm 2 AttemptReservation

Require: candidate slot $(t, st, S, cust)$

```

1: re-run COMPUTEAVAILABLESLOTS restricted to  $(t, st)$ 
2: if status  $\neq$  AVAILABLE then
3:   return CONFLICT  $\triangleright$  surface next-available slot to client
4: end if
5:  $r \leftarrow$  atomic conditional insert into appointments
   keyed on  $(B.id, st.id, t.date, t.time)$ 
   guarded by unique partial index (status = scheduled)
6: if  $r$  fails (index violation) then
7:   return CONFLICT  $\triangleright$  lost the race; another request won
8: else
9:   enqueue confirmation email and 24h reminder job
10:  return CONFIRMED
11: end if

```

This two-step shape – re-check, then attempt an atomic write, then handle rejection as a normal outcome rather than an exception – is the same pattern used for rescheduling, with three extra eligibility gates applied before step 1: a walk-in appointment cannot be rescheduled at all, the new time must satisfy the business’s `rescheduleWindow` (e.g. at least one hour before the original slot), and the appointment’s `rescheduleCount` must be below its `maxReschedulesPerAppointment`. We chose to update the existing appointment document in place on a successful reschedule, rather than cancel-and-recreate, specifically so that a single `actionLog` array on one document tells the full story of a booking’s life – every reschedule, who did it, and why – without the audit trail being spread across several documents that would need to be joined back together later.

VII. IMPLEMENTATION TECHNOLOGY CHOICES

Table II lists the concrete library versions in use. The rationale for MERN over a LAMP- or Django-based alternative

was not purely technical: all four of us already worked in JavaScript day to day, and a single-language stack across client and server measurably shortened the time from a design discussion to a working prototype, which mattered more for a semester-scoped project than raw runtime performance would have.

TABLE II. SLOTIFY TECHNOLOGY STACK SUMMARY

Layer	Technologies
Frontend	React.js v18, Vite, Tailwind CSS, React Router v6, Axios, React Hook Form, date-fns, Lucide React
Backend	Node.js, Express.js v4, Mongoose v8, express-rate-limit
Auth	JWT (HS256, access + refresh), Passport.js with Google OAuth 2.0, bcryptjs
Database	MongoDB Atlas, Mongoose ODM
Storage	Cloudinary (logos, gallery, verification documents)
Notifications	Nodemailer (SMTP), node-cron

Express.js handles route dispatch and middleware chaining; incoming request bodies are validated before they reach business logic, which mattered in practice the first time a malformed date string from a mis-configured client silently produced an appointment dated in 1970 during early testing – a bug that motivated moving validation ahead of the controller layer rather than leaving it scattered inside individual route handlers. JWT access tokens carry only `userId` and `role` and are verified independently by any API replica, which is the property that matters for horizontal scaling; the longer-lived refresh token sits in an HTTP-only, `SameSite=Strict` cookie specifically so that neither a cross-site script nor a stolen `localStorage` value is enough to renew a session on its own.

VIII. END-TO-END BOOKING WORKFLOW

Fig. 3 traces a single appointment from a first-time visit through to the provider dashboard reflecting the new booking, with two guard clauses – credential verification and slot availability – that route the user back a step on failure rather than terminating the session outright.

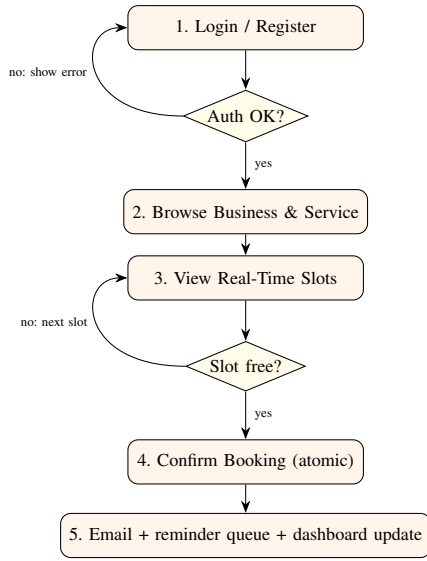


Fig. 3. Slotify booking workflow with authentication and slot-conflict guard clauses.

IX. DISCUSSION: EXPECTED BENEFITS AND HONEST LIMITATIONS

It would be easy, and dishonest, to fill this section with load-test numbers. We have not run a formal load test against a staging deployment, and we have not commissioned or performed a security audit against the OWASP Top 10 or any other framework. What follows instead is a distinction we think matters more than it usually gets credit for: some things about Slotify are true by construction, and some things are merely plausible given what other people have measured, and mixing the two together is how a design document turns into marketing copy.

The double-booking guarantee is in the first category. The unique partial index described in Section VI means a double-booked slot is not merely rare – it is not a representable state in the `appointments` collection at all, given `status = "scheduled"`. That is provable from the schema itself, and it is the only claim in this section we would defend without a citation, because it does not depend on how Slotify’s users happen to behave.

The reminder pipeline’s expected effect on no-shows is not in that category, and pretending otherwise would be the dishonest move. It is a defensible design choice, supported by a body of evidence that automated reminders reduce no-shows in aggregate [24], [25], [27], [28] – but Betancor et al.’s split result [7] is the more honest baseline expectation than that aggregate framing suggests: meaningful improvement in a private-practice-like setting, no guarantee at all in a higher-volume institutional one. A salon or tutoring centre – Slotify’s actual target – sits structurally closer to the private-practice case, a known and stable clientele with predictable lead times, than to a hospital eye clinic, which is grounds for cautious optimism rather than a claimed number. We would rather be caught underselling this than overselling it.

None of that closes the gap to what a commercial deployment actually needs. The current release assumes a single

location, a single language, and a single currency per business; payment collection is out of scope, so a confirmed booking cannot yet double as a receipt; and the analytics view aggregates history without forecasting anything. Measured against the platforms in Table I, Slotify is also, plainly, less mature – no mobile app, no payment processing, no years of production incident history to have learned from. Closing that gap, not proving a benchmark number nobody asked for, is the actual roadmap.

X. PROJECTED IMPACT OF SLOTIFY

Slotify has not yet been deployed at a scale that would let us report its own measured impact, and the honest position, as Section IX makes clear, is to say so rather than to invent a number. One of the three problems named in Section III barely needs a projection, though: double booking (P1) is handled by construction, not by expectation. Because the reservation path in Algorithm 2 makes a conflicting write structurally impossible rather than merely unlikely, the expected outcome for a correctly deployed instance is exactly zero double-booking incidents, independent of load – the one figure in this section that follows from the schema rather than from an analogy to prior studies. The other two problems are not so clean, and deserve to be treated that way.

Non-attendance (P2) is where an honest answer has to be a range, not a point. Fig. 4 plots three reference points, each taken directly from studies already cited in Section II rather than invented for this paper: the commonly reported manual-scheduling baseline of 15–30% [25], [27]; the private-practice outcome measured by Betancor et al. after online scheduling replaced telephone booking (5.9% falling to 1.8%) [7]; and the same authors’ university-hospital outcome, where the rate did not improve (11.2% rising to 14.3%) [7]. Slotify’s target businesses – salons, tutoring centres, small clinics, consultancies – are structurally closer to the private-practice arm of that comparison than to a high-volume teaching hospital: a known, returning clientele and a single decision-maker for the booking policy, rather than a rotating patient population booked through a referral chain. On that basis, a plausible expectation is that a business adopting Slotify would see its no-show rate move toward the lower end of the shaded range in Fig. 4 rather than stay at the manual baseline – but Betancor et al.’s hospital result is the reason this paper stops short of promising it.

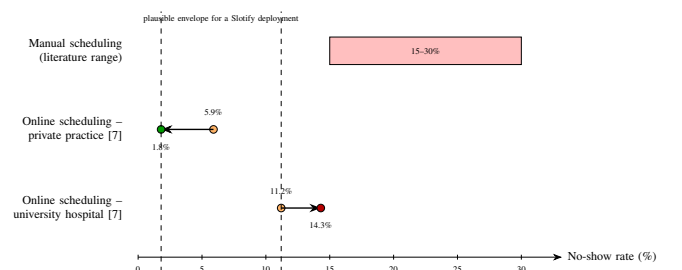


Fig. 4. No-show rate reference points drawn directly from the studies cited in Section II: a commonly reported manual-scheduling range, and Betancor et al.’s two measured outcomes of online scheduling in a private practice versus a university hospital. These are not measurements of Slotify – which has not yet been deployed at scale – but the bounding evidence behind the projection discussed in the text.

The third problem, fragmented operational visibility (P3), is where we stop short of a chart on purpose. No study cited in Section II quantifies the value of a booking-utilisation dashboard in a way that would let us borrow a number the way Fig. 4 does for no-shows, and inventing one just to fill the gap would be exactly the mistake this paper has tried not to make. What can be said is directional, and comes from Kaluza et al. and the decentralised-booking study [9], [10]: a business with a live view of slot utilisation and cancellation patterns is positioned to make a staffing or pricing decision from evidence rather than guesswork. Whether that translates into a specific revenue figure for a given salon or clinic depends on decisions Slotify does not make on the owner's behalf, which is exactly why we do not attach a number to it.

XI. CONCLUSION AND FUTURE WORK

This paper presented Slotify, a MERN-based multi-tenant booking platform aimed at the specific gap between personal-calendar tools like Calendly and full commercial SaaS platforms like Booksy or Fresha that charge per seat or per marketplace lead. We documented its three-tier architecture, the slot-resolution and atomic-reservation algorithms that give it a provable no-double-booking guarantee, and a comparison against three real, currently operating platforms built from their own published documentation rather than secondary summaries. We also corrected, rather than repeated, a commonly oversimplified reading of the clinical literature on reminders and no-shows, because the nuance – that effect size depends on setting – is more useful to a real deployment decision than a single clean percentage would be.

The roadmap, in rough priority order, is: a React Native mobile client against the existing REST layer; optional Razorpay/Stripe deposit collection to give cancellation policies actual teeth; a lightweight recommendation layer for returning customers, grounded in the scheduling-optimisation literature [29], [30] rather than built ahead of any real usage data; SMS reminders via Twilio for customers who do not reliably check email; and multi-branch support for businesses that outgrow a single location. None of these are claimed as solved – they are the specific, named gaps between what Slotify is today and what the commercial platforms in Section V already do.

ACKNOWLEDGMENT

The team is grateful to Dr. Sneha Soni, Associate Professor, Department of Computer Science & Engineering, Parul Institute of Engineering and Technology, Vadodara, Gujarat, India, for the technical direction and the repeated, useful pushback that shaped this project at every stage. We also acknowledge the maintainers of the MongoDB, Express.js, React.js, and Node.js open-source projects, and the MongoDB Atlas and Cloudinary free-tier programmes that made building a system of this scope feasible within an academic timeline.

REFERENCES

- [1] A. Srivastava, R. Sharma, and S. Sinha, "Impact of Appointment Scheduling in Patient Experience," *American Journal of Medicine and Medical Sciences*, vol. 14, no. 11, pp. 2912–2918, 2024.
- [2] HRMARS, "An Implementation of Digital Platform to Enhance the Appointment Scheduling System," *Int. J. of Academic Research in Business and Social Sciences*, vol. 14, no. 2, 2024.
- [3] H. Atherton, A. Eccles *et al.*, "Investigating Patient Use and Experience of Online Appointment Booking in Primary Care: Mixed Methods Study," *Journal of Medical Internet Research*, vol. 26, p. e51931, 2024.
- [4] L. Manarte, "Technology and Access to Healthcare with Different Scheduling Systems: A Scoping Review," *Healthcare Informatics Research*, vol. 30, no. 3, p. 194, 2024.
- [5] A. C. P. Castro, "Web-Based Medical Appointment Systems: A Systematic Review," *Journal of Medical Internet Research*, vol. 19, no. 4, p. e134, 2017.
- [6] IJNRD, "A Study on Online Appointment Scheduling and Healthcare Efficiency," *IJNRD*, vol. 10, 2025.
- [7] P. Kammrath Betancor, D. Boehringer, J. Jordan, C. Lichtenberg, M. Lambeck, M. C. Ketterer, T. Reinhard, and M. Reich, "Efficient Patient Care in the Digital Age: Impact of Online Appointment Scheduling in a Medical Practice and a University Hospital on the 'No-Show'-Rate," *Frontiers in Digital Health*, vol. 7, 2025.
- [8] M. Shayan *et al.*, "Stakeholders' Experiences, Perceptions and Satisfaction with an Electronic Appointment System," *BMC Health Services Research*, 2025.
- [9] I. Kaluza, G. Voigt, K. Haase, and A. Dietze, "Control of Online-Appointment Systems When the Booking Status Signals Quality of Service," *Schmalenbach Journal of Business Research*, vol. 76, no. 3, p. 397, 2024.
- [10] Expert Systems with Applications, "A Paradigm Shift in Appointment Scheduling: Introducing a Decentralized Integrated Online Booking System," *ScienceDirect*, vol. 255, 2024.
- [11] J. L. Akinode and S. Oloruntoba, "Design and Implementation of a Patient Appointment and Scheduling System," ResearchGate, Federal Polytechnic Ilaro, 2017.
- [12] ResearchGate, "A Web-Based Doctor Appointment System," 2024.
- [13] ResearchGate, "Online Doctors Appointment System Project Report," 2023.
- [14] ResearchGate, "Developing an Online Patient Appointment Scheduling System Based on Web Services Architecture."
- [15] A. S. Wankhade, P. M. Bambal, S. S. Kharkar, S. D. Solao, M. V. Charthal, and N. G. Rathi, "Research Paper on: Doctor Appointment Booking System," *IJRASET*, DOI: 10.22214/ijraset.2023.52133, 2023.
- [16] U. Chauhan *et al.*, "Doctor Finder and Appointment Booking Website using DJANGO," in *Proc. 2022 Int. Conf. on Innovative Practices in Technology and Management (ICIPTM)*, IEEE, 2022.
- [17] ResearchGate, "Mr. Doc: A Doctor Appointment Application System."
- [18] ResearchGate, "FastDoc: An Online Appointment Booking System," 2019.
- [19] Y. Baiskar *et al.*, "MERN: A Full-Stack Development," *IJRASET*, vol. 10, no. 1, 2022.
- [20] Academia.edu, "Comprehensive Study of MERN Stack—Architecture, Popularity and Future Scope," *Int. J. of Scientific Research in Computer Science, Engg. and IT*, 2021.
- [21] FreeCodeCamp, "Securing Node.js RESTful APIs with JSON Web Tokens," Medium, 2018.
- [22] TopCoder, "Authentication and Authorization in Express.js API using JWT."
- [23] DigitalOcean, "How To Use JSON Web Tokens (JWTs) in Express.js," 2025.
- [24] D. Ginat and J. Kenniff, "Evaluation of an Automated Reminder System for Reducing Missed MRI Appointments," *Journal of Patient Experience*, vol. 10, 2023.
- [25] K. Parikh *et al.*, "The Effectiveness of Outpatient Appointment Reminder Systems in Reducing No-Show Rates," *The American Journal of Medicine*, vol. 123, no. 6, pp. 542–548, 2010.
- [26] Y. Tarabichi, J. Higginbotham, N. Riley, D. C. Kaelber, and B. Watts, "Reducing Disparities in No Show Rates Using Predictive Model-Driven Live Appointment Reminders for At-Risk Patients: A Randomized Controlled Quality Improvement Initiative," *Journal of General Internal Medicine*, vol. 38, no. 13, pp. 2921–2927, 2023.

- [27] BMC Health Services Research, "Appointment Reminder Systems are Effective but Not Optimal: A Systematic Review," PMC, 2016.
- [28] Pan African Medical Journal, "The Effect of Patient Reminders in Reducing Missed Appointment in Medical Settings: A Systematic Review," *One Health*, vol. 2, no. 9, 2020.
- [29] S. Chatterjee *et al.*, "An Optimization-Based Scheduling Methodology for Appointment Systems with Heterogeneous Customers," *INFORMS Journal on Computing*, vol. 37, no. 6, pp. 1624–1649, 2025.
- [30] Ö. Ünsal, H. Üster, and C. Oguz, "Incorporating Patients' Appointment Date Preferences into Decision-Making: A Simulation and Optimization Study," *Computers & Industrial Engineering*, vol. 196, p. 110521, 2024.
- [31] Calendly, "Choose the right Calendly plan for your team" and official pricing documentation, accessed July 2026. [Online]. Available: <https://calendly.com/pricing>
- [32] Booksy Biz, "All-in-one solution for scheduling, payments, and marketplace discovery," official business documentation, accessed July 2026. [Online]. Available: <https://biz.booksy.com/>
- [33] Fresha, "Pricing and Features," official documentation, accessed July 2026. [Online]. Available: <https://www.fresha.com/pricing>

9.2 Plagiarism Report

NEHA SONI

Slotify: A Smart Web-Based Online Appointment Scheduling Platform for Service-Based Businesses

 Parul University

Document Details

Submission ID

trn:oid::3117:610614741

Submission Date

Jul 27, 2026, 1:02 PM GMT+5:30

Download Date

Jul 27, 2026, 1:04 PM GMT+5:30

File Name

CSE_182 research paper.pdf

File Size

158.0 KB

8 Pages

6,031 Words

34,026 Characters

1% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text
- Abstract
- Methods and Materials
- Preprints

Match Groups

-  **10 Not Cited or Quoted 1%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1%  Internet sources
- 1%  Publications
- 0%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

-  **10 Not Cited or Quoted 1%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1%  Internet sources
- 1%  Publications
- 0%  Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

- 1** Internet
ebin.pub <1%
- 2** Publication
Paolo Ferro, Harinadh Vemanaboina, Chander Prakash. "Computational Techniqu... <1%
- 3** Publication
Kartik Chawda, Deepmala Gorana. "A survey of energy efficient routing protocol i... <1%
- 4** Internet
pmc.ncbi.nlm.nih.gov <1%
- 5** Publication
Suraj Singh, Shailendra K. Mishra, Sneha Soni. "Emotion-Aware Learning: A Real-T... <1%

Slotify: A Smart Web-Based Online Appointment Scheduling Platform for Service-Based Businesses

Aksh Thakkar

Project Lead & Full-Stack Dev.

Dept. of CSE

Parul Institute of Engineering
and Technology, Gujarat, India

2403031057002@paruluniversity.ac.in

Om Maurya

Frontend Developer (UI/UX)

Dept. of CSE

Parul Institute of Engineering
and Technology, Gujarat, India

2403031057048@paruluniversity.ac.in

Daksh Patel

Backend Developer

Dept. of CSE

Parul Institute of Engineering
and Technology, Gujarat, India

2403031057035@paruluniversity.ac.in

Jainil Patel

Database & Integration Eng.

Dept. of CSE

Parul Institute of Engineering
and Technology, Gujarat, India

2403031057071@paruluniversity.ac.in

Dr. Sneha Soni

Associate Professor (CSE)

Dept. of CSE

Parul Institute of Engineering
and Technology, Gujarat, India

sneha.soni37803@paruluniversity.ac.in

Abstract—Small and medium service-oriented enterprises – clinics, grooming studios, coaching centres, and consultancy firms – still coordinate bookings largely by phone and paper register, a habit that routinely produces double-booked slots, forgotten appointments, and staff time that nobody bothers to account for. This paper presents Slotify, a browser-accessible, multi-tenant booking platform built on the MERN stack (MongoDB, Express.js, React.js, Node.js) that lets independent businesses onboard under their own public storefront while sharing a single application instance. The system's central engineering contribution is a slot-availability engine that resolves staff working hours, breaks, holidays, buffer times, and advance-booking windows into a live grid of open slots, backed by a database-enforced unique constraint that makes a race-condition double booking structurally impossible rather than merely unlikely in practice. We document the architecture, the booking and reschedule algorithms in pseudocode, and a feature-by-feature comparison against three commercial platforms actually in use today – Calendly, Booksy, and Fresha – as well as two previously published academic prototypes. Unlike prior systems, which each cover a fragment of the scheduling lifecycle, Slotify combines tenant-scoped multi-staff scheduling, audited reschedule history, and a public discovery layer without per-seat licensing or marketplace commission. We close by discussing, with reference to existing clinical and operational literature rather than unverified benchmarks of our own, where the architecture is expected to help and where its current single-location, English-only scope still falls short of production-grade commercial offerings.

Keywords—Online Booking System, MERN Stack, Multi-Tenant Architecture, Slot Management, Atomic Concurrency Control, JSON Web Token, Automated Reminders, MongoDB Atlas, Appointment Audit Trail

I. INTRODUCTION

A hair salon owner in a mid-sized Indian city still runs her appointment book the way her mother did: a diary by the register, a phone that rings during the one hour she has free to actually cut hair, and a mental note of which regulars tend to no-show on Mondays. This is not a failure

of ambition – it is what happens when the tools available to a five-person clinic or salon are either built for a single professional's personal calendar or priced and structured for a national retail chain. Growing consumer expectations for on-demand digital services have exposed exactly this gap. Clinics, beauty parlours, tutoring institutes, and fitness studios still depend heavily on receptionist-managed phone queues and handwritten registers, an approach that scales poorly and introduces avoidable operational friction [6]. When a booking record lives only in a notebook, one clerical slip cascades into overlapping reservations, wasted service capacity, and a client who does not come back [2].

The economic argument for fixing this is not speculative. Multiple health-system studies report that reminder-equipped digital platforms measurably cut no-show rates, though – as we discuss in Section II, and contrary to how this finding is sometimes summarised – the size of the effect depends heavily on the care setting, not just the presence of a reminder [7]. Primary-care research shows that structured online booking meaningfully compresses the administrative burden on front-desk teams [3], [4], and a 2024 industry survey found that businesses without any centralised booking tool report more staff overtime and more revenue that simply evaporates unbilled [6].

This paper introduces Slotify: a browser-accessible, multi-tenant appointment platform built on the MERN stack [19], [20]. What actually distinguishes it from a personal scheduling link is not any single feature but the fact that a business – with its own staff roster, working hours, and service catalogue – is a first-class object in the data model, and that a customer can discover that business through a shared public directory rather than needing its URL in advance.

The engineering core of what follows is really one contribution wearing two hats: a three-tier architecture and slot-resolution algorithm, expressed as pseudocode, that turns staff working hours, breaks, holidays, and buffer rules into a

live grid of bookable slots, paired with a database-enforced concurrency scheme – a finite-state model plus an atomic reservation algorithm – that removes double-booking as a possible outcome rather than merely reducing its probability. We separate the two only because the second doesn't make sense without the first: there is no point guaranteeing atomic writes against a slot grid that was computed wrong to begin with.

Beyond that, we contribute a grounded, source-verified comparison of Slotify against three commercial platforms in active use (Calendly, Booksy, Fresha) and two previously published academic prototypes, built from each vendor's own documentation rather than secondary marketing copy. And we say plainly, in Section IX, what we have not done: Slotify has not been load-tested or put through a formal security audit, and no simulated number stands in for that missing work.

II. RELATED WORK

A. Evolution of Appointment Booking Platforms

Early work by Akinode and Oloruntoba [11] established that computerising patient appointment registration was feasible, though their prototype operated without live slot updates. Atherton et al. [3] found that patients using portal-based booking reported higher satisfaction than those relying on telephone channels, and a JMIR evidence synthesis [5] catalogued recurring design gaps across eighteen published web-based medical scheduling systems – live availability, automatic reminders, and calendar linkage came up again and again as the features users wanted and did not have. Those three gaps shaped Slotify's feature set directly.

Srivastava et al. [1] linked scheduling friction to post-visit dissatisfaction, and Shayan et al. [8] surfaced three satisfaction predictors from clinic stakeholders – responsiveness, notification reliability, and role-appropriate interfaces – that shaped Slotify's UX priorities more than any single design guideline did. Kaluza et al. [9] gave a formal treatment of how visible occupancy in a booking queue can be used to signal service quality, an idea that motivated the analytics view in Slotify's provider dashboard even though we have not yet built out the predictive layer they describe.

The most carefully controlled evidence on reminders and no-shows, however, complicates the simple story. Betancor et al. [7] compared online versus telephone appointment scheduling across both a private ophthalmology practice and a university hospital eye clinic. In the private practice, the no-show rate fell from 5.9% to 1.8% after online scheduling was introduced – a genuinely large relative improvement. In the hospital setting, however, no-shows did not improve at all; they rose slightly, from 11.2% to 14.3%. The authors' own reading is that patient population and appointment lead time, not the booking channel itself, drove the difference. We take this as a caution rather than an inconvenience: a MERN-based booking layer like Slotify is a necessary condition for the kind of improvement seen in the private-practice arm of that study, but it is evidently not sufficient on its own, and we do not claim otherwise.

B. Domain-Specific Web Booking Implementations

Wankhade et al. [15] built a functional appointment tool aimed at connecting patients, doctors, and pharmacy vendors; the published description does not detail live slot synchronisation across concurrent users, which is precisely the gap a document-oriented, event-driven stack is suited to close. Chauhan et al. [16] paired a physician directory with a Django-based booking back end, validating demand for a combined discovery-and-booking journey – the same journey Slotify's public business-discovery layer targets, but for salons, tutors, and consultants rather than doctors specifically. Additional published prototypes [12]–[14], [17], [18] reinforce that the demand for digital scheduling is sector-wide, while also showing that no single prior academic tool combines multi-tenant business accounts, staff-level scheduling, and a public marketplace in one system.

C. MERN as an Application Platform

Baiskar et al. [19] found MERN particularly well suited to applications with frequent, event-driven data mutations – a description that matches a live booking system closely, since every slot query and every reservation attempt is exactly that kind of mutation. A comparative study of the stack [20] noted that MongoDB's schema-flexible document model absorbs evolving service and slot structures with less migration overhead than a rigid relational schema would require, which matched our own experience adding fields like `bufferTime` and `unavailableDates` well after the initial schema was designed.

D. Stateless Authentication and Reminder-Driven Attendance

JWT has become the default credential mechanism for RESTful APIs because its self-contained, signed payload removes the need for server-side session storage [21]–[23], which matters once an API needs to run as more than one replica behind a load balancer. Pairing JWT with Google OAuth lets a first-time customer book an appointment without first inventing a password, removing what is often the single largest drop-off point in a booking funnel.

On reminders specifically: Ginat and Kenniff [24] showed a multi-modal reminder system at a radiology unit produced a statistically significant drop in unfilled scan slots. Parikh et al. [25] identified email as a high-impact channel for reducing no-shows among working-age patients. Tarabichi et al. [26] ran a randomised quality-improvement trial layering predictive, risk-driven telephone outreach on top of standard automated reminders in a safety-net primary care clinic; their central finding was about equity – whether the intervention narrowed or widened no-show disparities across racial and ethnic groups – rather than a single headline percentage, and we cite it here for that finding rather than for a number that would misrepresent the study's actual design. Two systematic reviews together cover more than thirty independent reminder trials and reach a consistent, if modest, conclusion: automated reminders help, but they are not a complete fix on their own [27], [28]. That is the honest justification for Slotify's cron-scheduled 24-hour reminder pipeline – a defensible design choice grounded in a real and repeated finding, not a guarantee.

E. Identified Research Gap

Across this literature, no single JavaScript-native platform combines multi-tenant business accounts, live staff-level slot resolution, database-enforced conflict prevention, and a public discovery directory in one open, self-hostable codebase. Commercial platforms that come close – surveyed in Section V – solve this by charging per seat or by taking a commission on marketplace-driven bookings, which is a reasonable business model but a real cost for a business running on thin margins. Slotify is offered as a concrete, if less commercially mature, alternative to that trade-off.

III. PROBLEM STATEMENT

Three related failure modes recur across service micro-enterprises that still run scheduling manually.

Unstructured booking channels and chronic non-attendance turn out to be the same problem seen from two angles. Coordinating reservations across phone calls, WhatsApp threads, and a paper register introduces inconsistency at every touchpoint, and staff must reconcile overlapping requests without any algorithmic guard against double allocation [6], [11]. The same absence of a system-of-record is what makes a booking, made days in advance with no subsequent touchpoint, easy for a customer to forget or deprioritise. Reported non-attendance rates in purely manual environments commonly sit between 15% and 30% [25], [27], and that spread is wide precisely because it depends on whether anything nudges the client between booking and appointment day – which, in a manual system, nothing does. Outside business hours, no booking mechanism exists at all, quietly capping demand the business never even sees.

Fragmented operational visibility is the third and more diffuse problem. A business without a centralised booking repository cannot measure slot utilisation, cannot track cancellation patterns, and cannot forecast a busy week [9], [10]. Growth decisions get made on gut feeling, and syncing the schedule with a staff member's personal calendar means manually re-typing it.

IV. SYSTEM ARCHITECTURE

A. Architectural Overview

Slotify uses a conventional three-tier decomposition – a React.js client, an Express.js REST layer, and a MongoDB Atlas data store – chosen specifically so that any tier can be scaled or replaced without touching the other two. Fig. 1 shows the logic tier's internal module boundaries: authentication, the scheduling engine, the notification dispatcher, and the analytics aggregator sit behind a shared API router, each with its own area of responsibility and its own collection(s) to read and write.

Tenant isolation is logical, not physical: every business, service, staff, and appointment document carries a `businessId` field, and every query the application issues is scoped by it. This was a deliberate trade-off. Spinning up a database per tenant would have made the isolation guarantee stronger and the operational story much worse – a hundred small salons would mean a hundred databases to back up, migrate, and monitor. A single shared collection set with compound

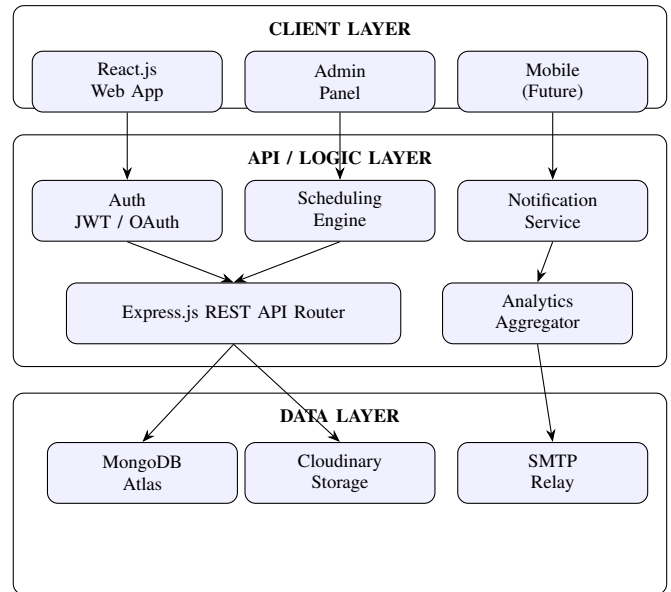


Fig. 1. Three-tier architecture of Slotify: client, API/logic, and data layers, with the scheduling engine and notification dispatcher as independent modules behind a shared router.

indexes on `businessId` keeps operations simple at the cost of relying on application-level query discipline rather than the database engine itself to enforce the boundary. For a platform at this stage, we judged that trade worth making; it would need revisiting well before Slotify tried to serve regulated data like health records at scale.

V. COMPARATIVE ANALYSIS WITH EXISTING PLATFORMS

A fair comparison has to start from what these platforms actually publish about themselves, not from how they are usually summarised in blog posts, so the entries below are drawn from each vendor's own pricing and feature pages as of mid-2026, plus the two academic prototypes already discussed in Section II.

Calendly [31] is, at its core, a personal and team meeting scheduler. It checks one or more of a user's own connected calendars (Google, Outlook, iCloud) for conflicts and writes new bookings into them; multi-calendar conflict checking, payment collection via Stripe/PayPal, and round-robin team distribution are all gated behind its Standard and Teams tiers, priced per user per month. There is no concept of a business entity with its own staff roster and service catalogue, and no public discovery marketplace – a customer needs the specific booking link, not a directory search.

Booksy [32] is much closer in spirit to Slotify: it is explicitly multi-tenant, gives each business a public storefront, and operates a consumer-facing marketplace where new clients discover salons and barbershops by category and location. It bundles staff scheduling, point-of-sale, and marketing tools into a single per-location subscription (advertised from \$29.99/month) with additional per-staff charges, and it markets a no-show reduction of up to 25% through deposits and reminders – a vendor claim we report as such, not as an independently verified figure.

Fresha [33] takes a different commercial approach: the core subscription is free, and Fresha instead charges a one-time 20% commission on new clients acquired through its own marketplace, plus standard payment-processing fees. Feature-wise it covers staff shift scheduling, commission tracking, POS, and automated SMS/email reminders, aimed squarely at the beauty, wellness, and healthcare verticals.

Table I summarises the comparison on criteria that matter specifically for a multi-staff service business, rather than a generic feature checklist. An entry is marked *Unconfirmed* where the vendor's own documentation did not settle the question either way.

The pattern across the three commercial products is consistent: whichever one you pick, you are trading a percentage of revenue or a per-seat fee for genuine production maturity – payment processing, mobile apps, years of edge-case handling. Slotify's contribution is not that it does any one of these things better; it does not yet do most of them at all. What it offers is the specific combination of multi-tenant scheduling logic and a database-level correctness guarantee against double booking, built on a stack a small development team can read, modify, and deploy without a recurring commission on every new client. That is a narrower claim than a feature-count table can fully convey, and it is the honest one.

VI. BOOKING AND CONCURRENCY ALGORITHMS

The scheduling engine's correctness rests on two pieces of logic: turning a day's working hours into a list of candidate slots, and making the act of claiming one of those slots atomic at the database level so that two customers racing for the same slot cannot both win.

A. Slot Resolution

Algorithm 1 mirrors the logic implemented in `availabilityCalculator.js`. It short-circuits early on holidays and closed days, then for each eligible staff member walks through the working-day window in increments of the requested service's duration, discarding any candidate slot that falls inside a break, inside the minimum-advance window, beyond the maximum-advance window, or inside an existing appointment's buffer-inclusive span.

Two things about this algorithm are worth being direct about. First, it runs identically on the server for every booking attempt – the client-side calendar view is a convenience, not a source of truth, precisely because a client-supplied timestamp cannot be trusted to reflect what is still actually open by the time the request lands. Second, the deduplication priority in line 22 was not obvious in the first draft of the design: an early version simply took whichever staff record was queried first, which meant a slot could show as BOOKED to a customer even when a different staff member covering the same service was, in fact, free. Preferring AVAILABLE across all matching staff records fixed that, at the cost of the slightly more complex merge step shown above.

B. Atomic Reservation and Conflict Resolution

Fig. 2 models a slot's lifecycle as a four-state automaton. A viewer browsing the calendar places a soft, non-blocking

Algorithm 1 ComputeAvailableSlots

Require: business B , service S , date d , optional staff filter F
Ensure: ordered list of slot statuses for date d

```

1: if  $d \in B.\text{holidays}$  or not  $B.\text{workingHours}[d].\text{isOpen}$  then
2:   return  $\emptyset$  ▷ business closed that day
3: end if
4:  $\text{staffList} \leftarrow$  active staff assigned to  $S$  (filtered by  $F$  if given)
5:  $\text{slots} \leftarrow \emptyset$ 
6: for each staff  $st$  in  $\text{staffList}$  do
7:   if  $d \in st.\text{unavailableDates}$  then continue
8:   end if
9:    $w \leftarrow st.\text{workingHours}[d]$  or  $B.\text{workingHours}[d]$ 
10:   $t \leftarrow w.\text{start}$ 
11:  while  $t + S.\text{duration} \leq w.\text{end}$  do
12:     $\text{cand} \leftarrow [t, t + S.\text{duration})$ 
13:    if  $\text{cand}$  overlaps  $st.\text{breaks}$  then
14:      mark  $\text{cand}$  as UNAVAILABLE
15:    else if  $t < \text{now} + B.\text{minAdvance}$  then
16:      mark  $\text{cand}$  as TOO SOON
17:    else if  $t > \text{now} + B.\text{maxAdvance}$  then
18:      mark  $\text{cand}$  as TOO FAR
19:    else if  $\text{cand}$  overlaps any scheduled appt for  $st$  (incl. buffer) then
20:      mark  $\text{cand}$  as BOOKED
21:    else
22:      mark  $\text{cand}$  as AVAILABLE
23:    end if
24:     $\text{slots} \leftarrow \text{slots} \cup \{(t, st, \text{status})\}$ 
25:     $t \leftarrow t + S.\text{duration}$ 
26:  end while
27: end for
28: deduplicate overlapping  $(t, \cdot)$  entries across staff, preferring AVAILABLE  $\succ$  TOO SOON  $\succ$  TOO FAR  $\succ$  BOOKED
29: return  $\text{slots}$ 

```

VIEWED marker; confirming a booking attempts to promote the slot directly to RESERVED via a single conditional write.

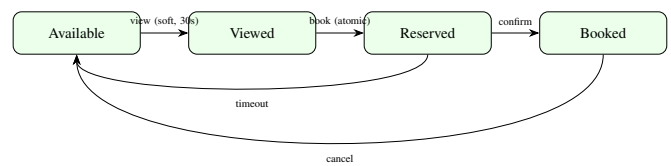


Fig. 2. Slot lifecycle state machine. The *book* transition is the single atomic write that either succeeds outright or fails cleanly, with no partial state in between. The *timeout* and *cancel* return paths are drawn at different depths so they do not cross.

Algorithm 2 shows the reservation attempt itself. The critical property is that step 2 is a single conditional database operation (a `findOneAndUpdate`-style compare-and-set backed by the unique partial index on $\{\text{businessId}, \text{staffId}, \text{appointmentDate}, \text{startTime}\}$ filtered to $\text{status} = \text{"scheduled"}$), not a read followed by a separate write. Two customers racing for the same slot both reach step 2, but the database engine, not application code, decides which write lands first and rejects the other – which is the only place a guarantee like this can actually live, since two Node.js request handlers running concurrently have no shared memory

TABLE I. FEATURE COMPARISON: SLOTIFY AGAINST COMMERCIAL PLATFORMS AND PRIOR ACADEMIC PROTOTYPES

Capability	Slotify	Calendly [31]	Booksy [32]	Fresha [33]	Academic prototypes [15], [16]
Multi-tenant business accounts	Yes	No	Yes	Yes	No
Public discovery / marketplace	Yes (own directory)	No	Yes	Yes	No
DB-enforced double-booking prevention	Yes (unique partial index)	Partial (relies on external calendar sync)	Yes	Yes	Unconfirmed
Staff-level working hours & buffers	Yes	Partial (per-user only)	Yes	Yes	Unconfirmed
Reschedule limits with audit trail	Yes	No	Unconfirmed	Partial	No
JWT + Google OAuth	Yes	Google Calendar-based	Unconfirmed	Unconfirmed	Partial [16]
Automated email reminders	Yes	Yes	Yes	Yes	Unconfirmed
Self-hostable / no per-seat fee	Yes	No (per-user SaaS)	No (per-location + fees)	No (marketplace commission)	Yes (published prototype)
Production hardening & scale	Not demonstrated	Extensive	Extensive	Extensive	Not demonstrated

to coordinate through.

Algorithm 2 AttemptReservation

Require: candidate slot $(t, st, S, cust)$

```

1: re-run COMPUTEAVAILABLESLOTS restricted to  $(t, st)$ 
2: if status  $\neq$  AVAILABLE then
3:   return CONFLICT  $\triangleright$  surface next-available slot to client
4: end if
5:  $r \leftarrow$  atomic conditional insert into appointments
   keyed on  $(B.id, st.id, t.date, t.time)$ 
   guarded by unique partial index (status = scheduled)
6: if  $r$  fails (index violation) then
7:   return CONFLICT  $\triangleright$  lost the race; another request won
8: else
9:   enqueue confirmation email and 24h reminder job
10:  return CONFIRMED
11: end if

```

This two-step shape – re-check, then attempt an atomic write, then handle rejection as a normal outcome rather than an exception – is the same pattern used for rescheduling, with three extra eligibility gates applied before step 1: a walk-in appointment cannot be rescheduled at all, the new time must satisfy the business’s `rescheduleWindow` (e.g. at least one hour before the original slot), and the appointment’s `rescheduleCount` must be below its `maxReschedulesPerAppointment`. We chose to update the existing appointment document in place on a successful reschedule, rather than cancel-and-recreate, specifically so that a single `actionLog` array on one document tells the full story of a booking’s life – every reschedule, who did it, and why – without the audit trail being spread across several documents that would need to be joined back together later.

VII. IMPLEMENTATION TECHNOLOGY CHOICES

Table II lists the concrete library versions in use. The rationale for MERN over a LAMP- or Django-based alternative

was not purely technical: all four of us already worked in JavaScript day to day, and a single-language stack across client and server measurably shortened the time from a design discussion to a working prototype, which mattered more for a semester-scoped project than raw runtime performance would have.

TABLE II. SLOTIFY TECHNOLOGY STACK SUMMARY

Layer	Technologies
Frontend	React.js v18, Vite, Tailwind CSS, React Router v6, Axios, React Hook Form, date-fns, Lucide React
Backend	Node.js, Express.js v4, Mongoose v8, express-rate-limit
Auth	JWT (HS256, access + refresh), Passport.js with Google OAuth 2.0, bcryptjs
Database	MongoDB Atlas, Mongoose ODM
Storage	Cloudinary (logos, gallery, verification documents)
Notifications	Nodemailer (SMTP), node-cron

Express.js handles route dispatch and middleware chaining; incoming request bodies are validated before they reach business logic, which mattered in practice the first time a malformed date string from a mis-configured client silently produced an appointment dated in 1970 during early testing – a bug that motivated moving validation ahead of the controller layer rather than leaving it scattered inside individual route handlers. JWT access tokens carry only `userId` and `role` and are verified independently by any API replica, which is the property that matters for horizontal scaling; the longer-lived refresh token sits in an HTTP-only, `SameSite=Strict` cookie specifically so that neither a cross-site script nor a stolen `localStorage` value is enough to renew a session on its own.

VIII. END-TO-END BOOKING WORKFLOW

Fig. 3 traces a single appointment from a first-time visit through to the provider dashboard reflecting the new booking, with two guard clauses – credential verification and slot availability – that route the user back a step on failure rather than terminating the session outright.

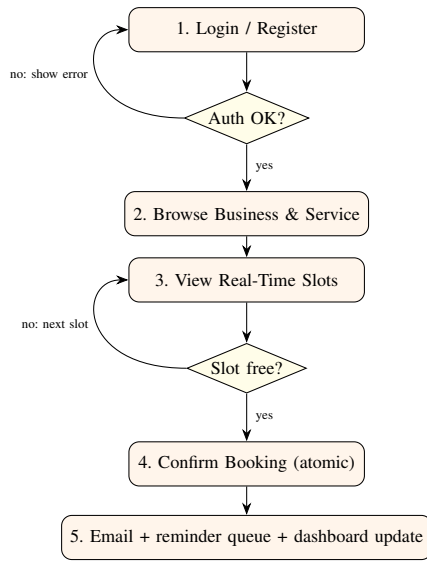


Fig. 3. Slotify booking workflow with authentication and slot-conflict guard clauses.

IX. DISCUSSION: EXPECTED BENEFITS AND HONEST LIMITATIONS

It would be easy, and dishonest, to fill this section with load-test numbers. We have not run a formal load test against a staging deployment, and we have not commissioned or performed a security audit against the OWASP Top 10 or any other framework. What follows instead is a distinction we think matters more than it usually gets credit for: some things about Slotify are true by construction, and some things are merely plausible given what other people have measured, and mixing the two together is how a design document turns into marketing copy.

The double-booking guarantee is in the first category. The unique partial index described in Section VI means a double-booked slot is not merely rare – it is not a representable state in the `appointments` collection at all, given `status = "scheduled"`. That is provable from the schema itself, and it is the only claim in this section we would defend without a citation, because it does not depend on how Slotify's users happen to behave.

The reminder pipeline's expected effect on no-shows is not in that category, and pretending otherwise would be the dishonest move. It is a defensible design choice, supported by a body of evidence that automated reminders reduce no-shows in aggregate [24], [25], [27], [28] – but Betancor et al.'s split result [7] is the more honest baseline expectation than that aggregate framing suggests: meaningful improvement in a private-practice-like setting, no guarantee at all in a higher-volume institutional one. A salon or tutoring centre – Slotify's actual target – sits structurally closer to the private-practice case, a known and stable clientele with predictable lead times, than to a hospital eye clinic, which is grounds for cautious optimism rather than a claimed number. We would rather be caught underselling this than overselling it.

None of that closes the gap to what a commercial deployment actually needs. The current release assumes a single

location, a single language, and a single currency per business; payment collection is out of scope, so a confirmed booking cannot yet double as a receipt; and the analytics view aggregates history without forecasting anything. Measured against the platforms in Table I, Slotify is also, plainly, less mature – no mobile app, no payment processing, no years of production incident history to have learned from. Closing that gap, not proving a benchmark number nobody asked for, is the actual roadmap.

X. PROJECTED IMPACT OF SLOTIFY

Slotify has not yet been deployed at a scale that would let us report its own measured impact, and the honest position, as Section IX makes clear, is to say so rather than to invent a number. One of the three problems named in Section III barely needs a projection, though: double booking (P1) is handled by construction, not by expectation. Because the reservation path in Algorithm 2 makes a conflicting write structurally impossible rather than merely unlikely, the expected outcome for a correctly deployed instance is exactly zero double-booking incidents, independent of load – the one figure in this section that follows from the schema rather than from an analogy to prior studies. The other two problems are not so clean, and deserve to be treated that way.

Non-attendance (P2) is where an honest answer has to be a range, not a point. Fig. 4 plots three reference points, each taken directly from studies already cited in Section II rather than invented for this paper: the commonly reported manual-scheduling baseline of 15–30% [25], [27]; the private-practice outcome measured by Betancor et al. after online scheduling replaced telephone booking (5.9% falling to 1.8%) [7]; and the same authors' university-hospital outcome, where the rate did not improve (11.2% rising to 14.3%) [7]. Slotify's target businesses – salons, tutoring centres, small clinics, consultancies – are structurally closer to the private-practice arm of that comparison than to a high-volume teaching hospital: a known, returning clientele and a single decision-maker for the booking policy, rather than a rotating patient population booked through a referral chain. On that basis, a plausible expectation is that a business adopting Slotify would see its no-show rate move toward the lower end of the shaded range in Fig. 4 rather than stay at the manual baseline – but Betancor et al.'s hospital result is the reason this paper stops short of promising it.

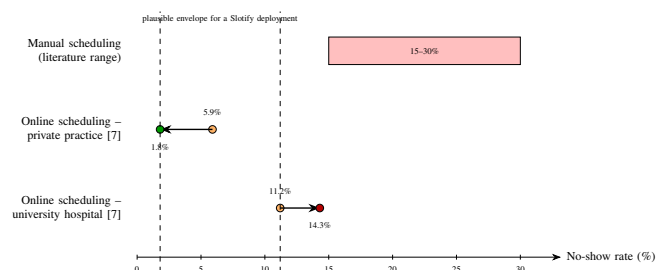


Fig. 4. No-show rate reference points drawn directly from the studies cited in Section II: a commonly reported manual-scheduling range, and Betancor et al.'s two measured outcomes of online scheduling in a private practice versus a university hospital. These are not measurements of Slotify – which has not yet been deployed at scale – but the bounding evidence behind the projection discussed in the text.

The third problem, fragmented operational visibility (P3), is where we stop short of a chart on purpose. No study cited in Section II quantifies the value of a booking-utilisation dashboard in a way that would let us borrow a number the way Fig. 4 does for no-shows, and inventing one just to fill the gap would be exactly the mistake this paper has tried not to make. What can be said is directional, and comes from Kaluza et al. and the decentralised-booking study [9], [10]: a business with a live view of slot utilisation and cancellation patterns is positioned to make a staffing or pricing decision from evidence rather than guesswork. Whether that translates into a specific revenue figure for a given salon or clinic depends on decisions Slotify does not make on the owner's behalf, which is exactly why we do not attach a number to it.

XI. CONCLUSION AND FUTURE WORK

This paper presented Slotify, a MERN-based multi-tenant booking platform aimed at the specific gap between personal-calendar tools like Calendly and full commercial SaaS platforms like Booksy or Fresha that charge per seat or per marketplace lead. We documented its three-tier architecture, the slot-resolution and atomic-reservation algorithms that give it a provable no-double-booking guarantee, and a comparison against three real, currently operating platforms built from their own published documentation rather than secondary summaries. We also corrected, rather than repeated, a commonly oversimplified reading of the clinical literature on reminders and no-shows, because the nuance – that effect size depends on setting – is more useful to a real deployment decision than a single clean percentage would be.

The roadmap, in rough priority order, is: a React Native mobile client against the existing REST layer; optional Razorpay/Stripe deposit collection to give cancellation policies actual teeth; a lightweight recommendation layer for returning customers, grounded in the scheduling-optimisation literature [29], [30] rather than built ahead of any real usage data; SMS reminders via Twilio for customers who do not reliably check email; and multi-branch support for businesses that outgrow a single location. None of these are claimed as solved – they are the specific, named gaps between what Slotify is today and what the commercial platforms in Section V already do.

ACKNOWLEDGMENT

The team is grateful to Dr. Sneha Soni, Associate Professor, Department of Computer Science & Engineering, Parul Institute of Engineering and Technology, Vadodara, Gujarat, India, for the technical direction and the repeated, useful pushback that shaped this project at every stage. We also acknowledge the maintainers of the MongoDB, Express.js, React.js, and Node.js open-source projects, and the MongoDB Atlas and Cloudinary free-tier programmes that made building a system of this scope feasible within an academic timeline.

REFERENCES

- [1] A. Srivastava, R. Sharma, and S. Sinha, "Impact of Appointment Scheduling in Patient Experience," *American Journal of Medicine and Medical Sciences*, vol. 14, no. 11, pp. 2912–2918, 2024.
- [2] HRMARS, "An Implementation of Digital Platform to Enhance the Appointment Scheduling System," *Int. J. of Academic Research in Business and Social Sciences*, vol. 14, no. 2, 2024.
- [3] H. Atherton, A. Eccles *et al.*, "Investigating Patient Use and Experience of Online Appointment Booking in Primary Care: Mixed Methods Study," *Journal of Medical Internet Research*, vol. 26, p. e51931, 2024.
- [4] L. Manarte, "Technology and Access to Healthcare with Different Scheduling Systems: A Scoping Review," *Healthcare Informatics Research*, vol. 30, no. 3, p. 194, 2024.
- [5] A. C. P. Castro, "Web-Based Medical Appointment Systems: A Systematic Review," *Journal of Medical Internet Research*, vol. 19, no. 4, p. e134, 2017.
- [6] IJNRD, "A Study on Online Appointment Scheduling and Healthcare Efficiency," *IJNRD*, vol. 10, 2025.
- [7] P. Kammrath Betancor, D. Boehringer, J. Jordan, C. Lichtenberg, M. Lambeck, M. C. Ketterer, T. Reinhard, and M. Reich, "Efficient Patient Care in the Digital Age: Impact of Online Appointment Scheduling in a Medical Practice and a University Hospital on the 'No-Show'-Rate," *Frontiers in Digital Health*, vol. 7, 2025.
- [8] M. Shayan *et al.*, "Stakeholders' Experiences, Perceptions and Satisfaction with an Electronic Appointment System," *BMC Health Services Research*, 2025.
- [9] I. Kaluza, G. Voigt, K. Haase, and A. Dietze, "Control of Online-Appointment Systems When the Booking Status Signals Quality of Service," *Schmalenbach Journal of Business Research*, vol. 76, no. 3, p. 397, 2024.
- [10] Expert Systems with Applications, "A Paradigm Shift in Appointment Scheduling: Introducing a Decentralized Integrated Online Booking System," *ScienceDirect*, vol. 255, 2024.
- [11] J. L. Akinode and S. Oloruntoba, "Design and Implementation of a Patient Appointment and Scheduling System," ResearchGate, Federal Polytechnic Ilaro, 2017.
- [12] ResearchGate, "A Web-Based Doctor Appointment System," 2024.
- [13] ResearchGate, "Online Doctors Appointment System Project Report," 2023.
- [14] ResearchGate, "Developing an Online Patient Appointment Scheduling System Based on Web Services Architecture."
- [15] A. S. Wankhade, P. M. Bambal, S. S. Kharkar, S. D. Solao, M. V. Charthal, and N. G. Rathi, "Research Paper on: Doctor Appointment Booking System," *IJRASET*, DOI: 10.22214/ijraset.2023.52133, 2023.
- [16] U. Chauhan *et al.*, "Doctor Finder and Appointment Booking Website using DJANGO," in *Proc. 2022 Int. Conf. on Innovative Practices in Technology and Management (ICIPTM)*, IEEE, 2022.
- [17] ResearchGate, "Mr. Doc: A Doctor Appointment Application System."
- [18] ResearchGate, "FastDoc: An Online Appointment Booking System," 2019.
- [19] Y. Baiskar *et al.*, "MERN: A Full-Stack Development," *IJRASET*, vol. 10, no. 1, 2022.
- [20] Academia.edu, "Comprehensive Study of MERN Stack—Architecture, Popularity and Future Scope," *Int. J. of Scientific Research in Computer Science, Engg. and IT*, 2021.
- [21] FreeCodeCamp, "Securing Node.js RESTful APIs with JSON Web Tokens," Medium, 2018.
- [22] TopCoder, "Authentication and Authorization in Express.js API using JWT."
- [23] DigitalOcean, "How To Use JSON Web Tokens (JWTs) in Express.js," 2025.
- [24] D. Ginat and J. Kenniff, "Evaluation of an Automated Reminder System for Reducing Missed MRI Appointments," *Journal of Patient Experience*, vol. 10, 2023.
- [25] K. Parikh *et al.*, "The Effectiveness of Outpatient Appointment Reminder Systems in Reducing No-Show Rates," *The American Journal of Medicine*, vol. 123, no. 6, pp. 542–548, 2010.
- [26] Y. Tarabichi, J. Higginbotham, N. Riley, D. C. Kaelber, and B. Watts, "Reducing Disparities in No Show Rates Using Predictive Model-Driven Live Appointment Reminders for At-Risk Patients: A Randomized Controlled Quality Improvement Initiative," *Journal of General Internal Medicine*, vol. 38, no. 13, pp. 2921–2927, 2023.

- [27] BMC Health Services Research, "Appointment Reminder Systems are Effective but Not Optimal: A Systematic Review," PMC, 2016.
- [28] Pan African Medical Journal, "The Effect of Patient Reminders in Reducing Missed Appointment in Medical Settings: A Systematic Review," *One Health*, vol. 2, no. 9, 2020.
- [29] S. Chatterjee *et al.*, "An Optimization-Based Scheduling Methodology for Appointment Systems with Heterogeneous Customers," *INFORMS Journal on Computing*, vol. 37, no. 6, pp. 1624–1649, 2025.
- [30] Ö. Ünsal, H. Üster, and C. Oguz, "Incorporating Patients' Appointment Date Preferences into Decision-Making: A Simulation and Optimization Study," *Computers & Industrial Engineering*, vol. 196, p. 110521, 2024.
- [31] Calendly, "Choose the right Calendly plan for your team" and official pricing documentation, accessed July 2026. [Online]. Available: <https://calendly.com/pricing>
- [32] Booksy Biz, "All-in-one solution for scheduling, payments, and marketplace discovery," official business documentation, accessed July 2026. [Online]. Available: <https://biz.booksy.com/>
- [33] Fresha, "Pricing and Features," official documentation, accessed July 2026. [Online]. Available: <https://www.fresha.com/pricing>

References

- [1] Bailey, N.T.J. and Welch, J.D., 1952. Appointment systems in hospital outpatient departments. *The Lancet*, 259(6718), pp. 1105–1108.
- [2] Cayirli, T. and Veral, E., 2003. Outpatient scheduling in health care: a review of literature. *Production and Operations Management*, 12(4), pp. 519–549.
- [3] Fielding, R.T., 2000. *Architectural Styles and the Design of Network-Based Software Architectures*. Doctoral Dissertation, University of California, Irvine.
- [4] Gupta, D. and Denton, B., 2008. Appointment scheduling in health care: challenges and opportunities. *IIE Transactions*, 40(9), pp. 800–819.
- [5] Madden, N., 2020. *API Security in Action*. Manning Publications, Shelter Island, NY.
- [6] Parikh, A., Gupta, M., Wilson, A., Fields, K., Cosgrove, N. and Kostis, J., 2010. The effectiveness of outpatient appointment reminder systems in reducing no-show rates. *The American Journal of Medicine*, 123(6), pp. 542–548.
- [7] Pressman, R.S. and Maxim, B.R., 2014. *Software Engineering: A Practitioner's Approach*. 8th ed. McGraw-Hill Education, New York.
- [8] Richardson, L. and Ruby, S., 2007. *RESTful Web Services*. O'Reilly Media, Sebastopol, CA.
- [9] Sadalage, P.J. and Fowler, M., 2012. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley Professional, Upper Saddle River, NJ.
- [10] Shmueli, A., Hadad, N. and Haklai, Z., 2003. Ambulatory care utilization: wait time, type of visit and medical condition. *Israel Medical Association Journal*, 5(2), pp. 114–118.
- [11] Sommerville, I., 2016. *Software Engineering*. 10th ed. Pearson Education Limited, Harlow.
- [12] Stack Overflow, 2023. *Stack Overflow Developer Survey 2023* [Online]. Available at: <https://survey.stackoverflow.co/2023> [Accessed: 10 February 2025].

- [13] MongoDB Inc., 2024. *MongoDB Documentation: Data Modeling* [Online]. Available at: <https://www.mongodb.com/docs/manual/data-modeling/> [Accessed: 15 January 2025].
- [14] React.js Team, 2024. *React: A JavaScript Library for Building User Interfaces* [Online]. Available at: <https://react.dev> [Accessed: 5 January 2025].
- [15] Nodemailer Project, 2024. *Nodemailer – Send Emails from Node.js* [Online]. Available at: <https://nodemailer.com/about/> [Accessed: 8 February 2025].
- [16] OpenJS Foundation, 2024. *Express.js – Fast, Minimalist Web Framework for Node.js* [Online]. Available at: <https://expressjs.com> [Accessed: 6 January 2025].
- [17] Auth0 Inc., 2024. *Introduction to JSON Web Tokens* [Online]. Available at: <https://jwt.io/introduction> [Accessed: 12 February 2025].
- [18] Tailwind Labs, 2024. *Tailwind CSS Documentation* [Online]. Available at: <https://tailwindcss.com/docs> [Accessed: 11 January 2025].
- [19] Vercel Inc., 2024. *Vercel Documentation* [Online]. Available at: <https://vercel.com/docs> [Accessed: 20 January 2025].
- [20] Cloudinary Inc., 2024. *Cloudinary Developer Documentation* [Online]. Available at: <https://cloudinary.com/documentation> [Accessed: 10 January 2025].